Software Engineering

Object-Oriented Design

Objectives

- To explain how a software design may be represented as a set of interacting objects that manage their own state and operations
- To describe the activities in the object-oriented design process
- To introduce various models that describe an object-oriented design
- To show how the UML may be used to represent these models

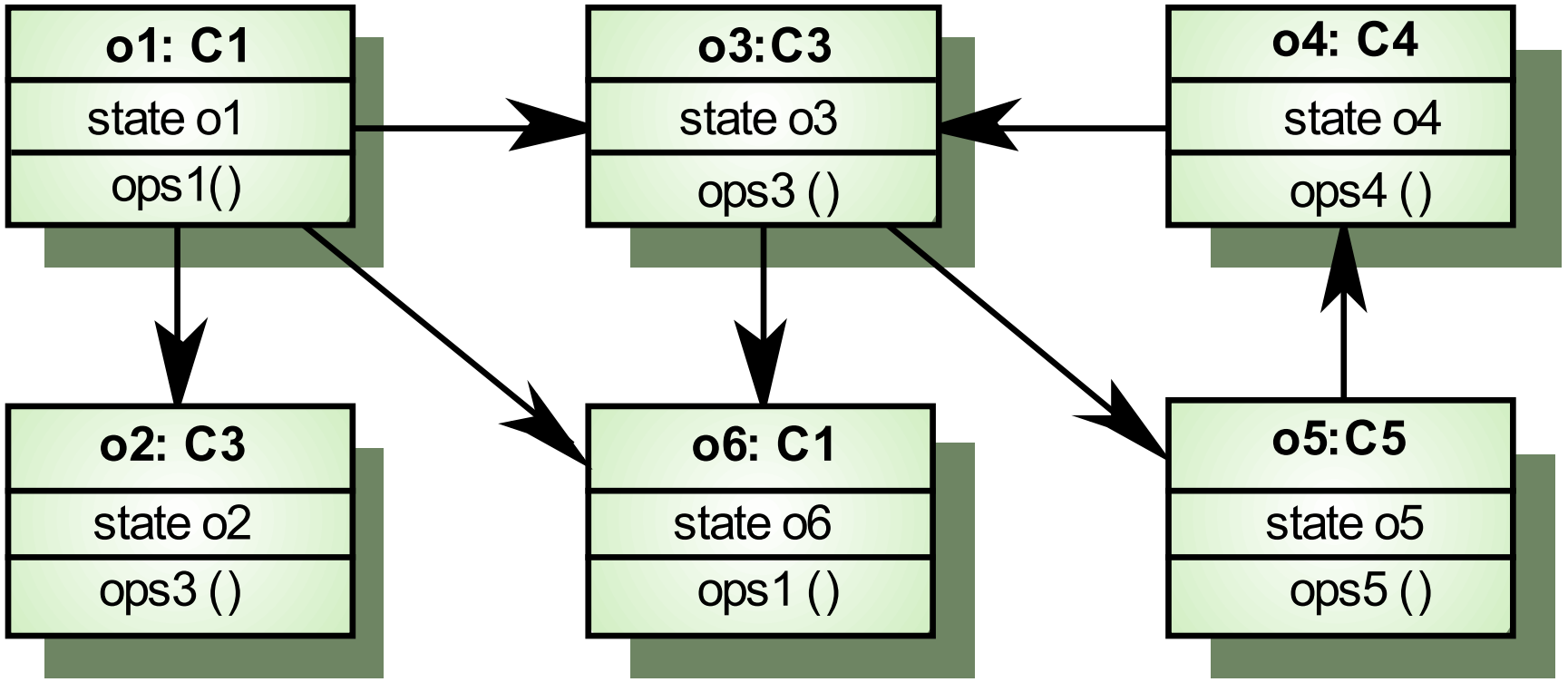
Object-oriented Design

- Designing systems using self-contained objects and object classes

Characteristics of OOD

- Objects are abstractions of real-world or system entities and manage themselves
- Objects are independent and encapsulate state and representation information.
- System functionality is expressed in terms of object services
- Shared data areas are eliminated. Objects communicate by message passing

Interacting objects



Advantages of OOD

- Easier maintenance. Objects may be understood as stand-alone entities
- Objects are appropriate reusable components
- For some systems, there may be an obvious mapping from real world entities to system objects

Object-oriented development

- Object-oriented analysis, design and programming are related but distinct
- OOA is concerned with developing an object model of the application domain
- OOD is concerned with developing an object-oriented system model to implement requirements
- OOP is concerned with realising an OOD using an OO programming language such as Java or C++

Objects and object classes

- Objects are entities in a software system which represent instances of real-world and system entities
- Object classes are templates for objects. They may be used to create objects
- Object classes may inherit attributes and services from other object classes

Objects

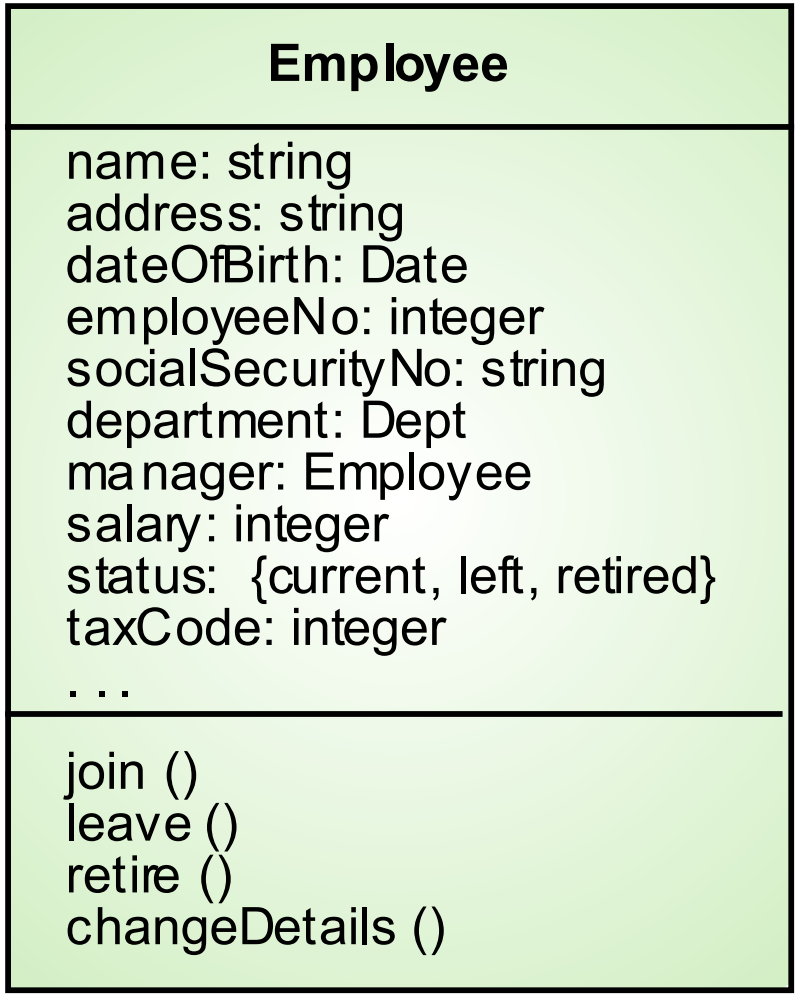
An **object** is an entity which has a state and a defined set of operations which operate on that state. The state is represented as a set of object attributes. The operations associated with the object provide services to other objects (clients) which request these services when some computation is required.

Objects are created according to some **object class** definition. An object class definition serves as a template for objects. It includes declarations of all the attributes and services which should be associated with an object of that class.

The Unified Modeling Language

- Several different notations for describing object-oriented designs were proposed in the 1980s and 1990s
- The Unified Modeling Language is an integration of these notations
- It describes notations for a number of different models that may be produced during OO analysis and design
- It is now a *de facto* standard for OO modelling

Employee object class (UML)



Object communication

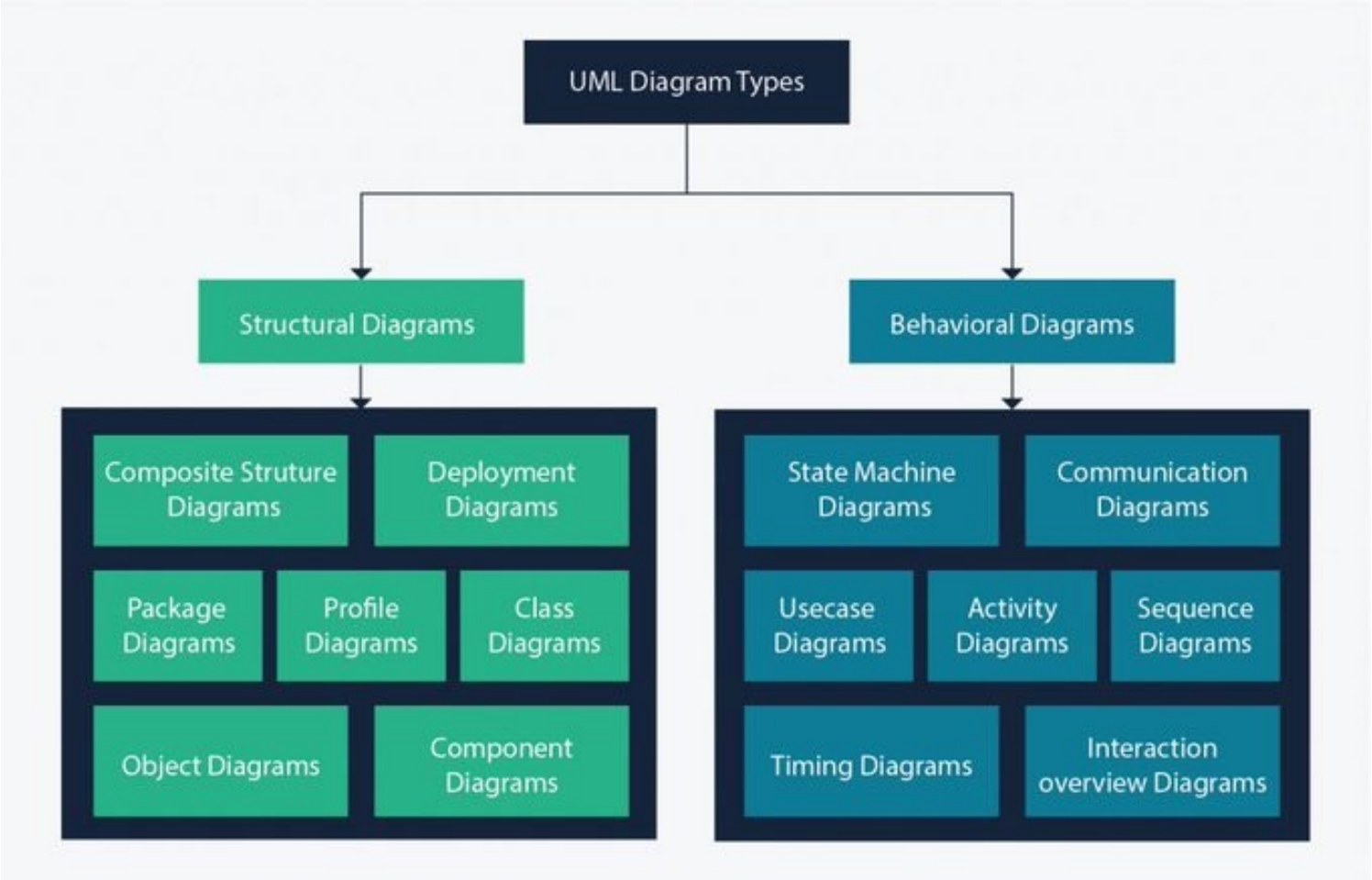
- Conceptually, objects communicate by message passing.
- Messages
  - The name of the service requested by the calling object.
  - Copies of the information required to execute the service and the name of a holder for the result of the service.
- In practice, messages are often implemented by procedure calls
  - Name = procedure name.
  - Information = parameter list.

Message examples

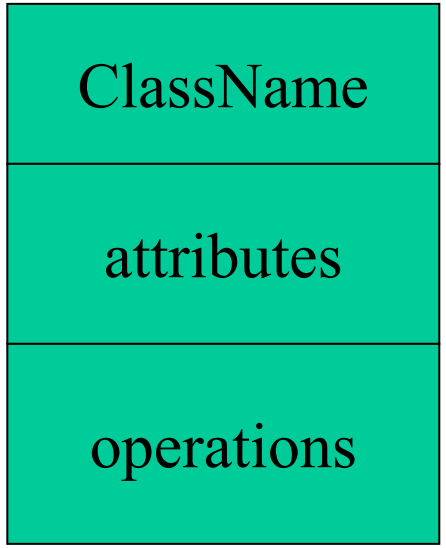
```
// Call a method associated with a buffer
// object that returns the next value
// in the buffer
        v = circularBuffer.Get () ;

// Call the method associated with a
// thermostat object that sets the
// temperature to be maintained
        thermostat.setTemp (20) ;
```

Unified Modeling Language (UML)



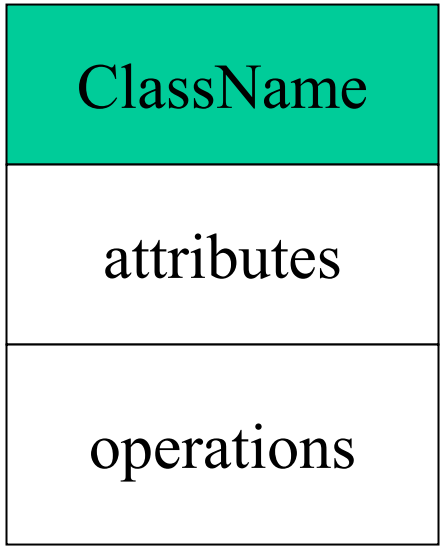
Classes



A *class* is a description of a set of objects that share the same attributes, operations, relationships, and semantics.

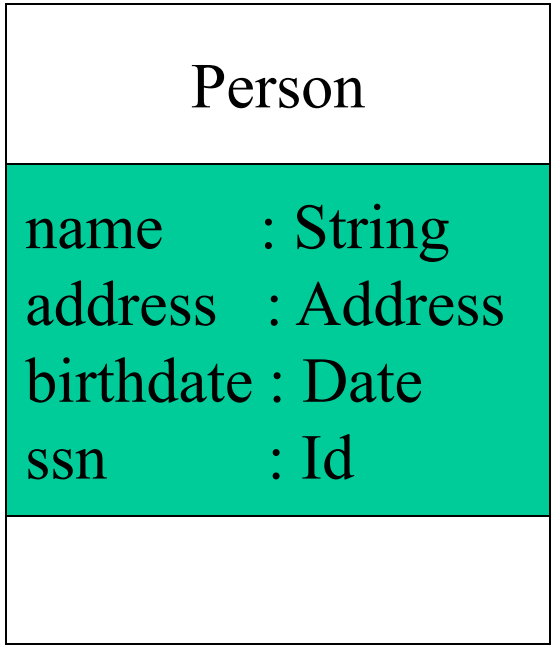
Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations in separate, designated compartments.

Class Names



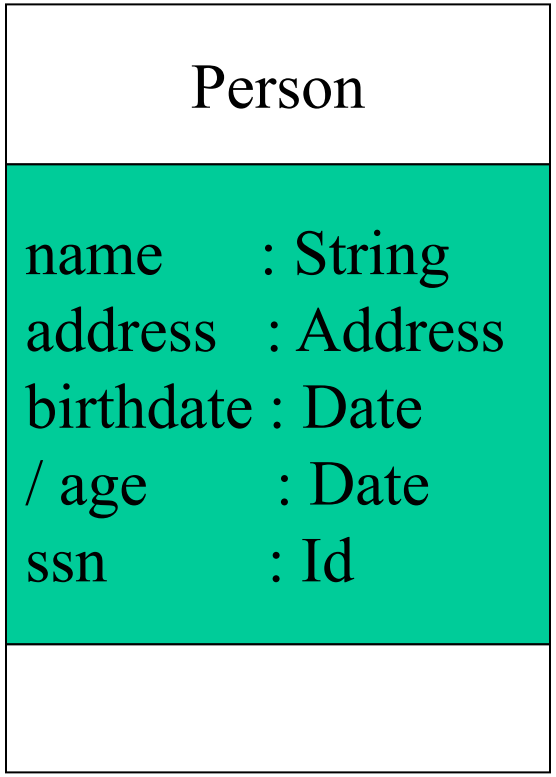
The name of the class is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.

Class Attributes



An *attribute* is a named property of a class that describes the object being modeled. In the class diagram, attributes appear in the second compartment just below the name-compartment.

Class Attributes (Cont'd)



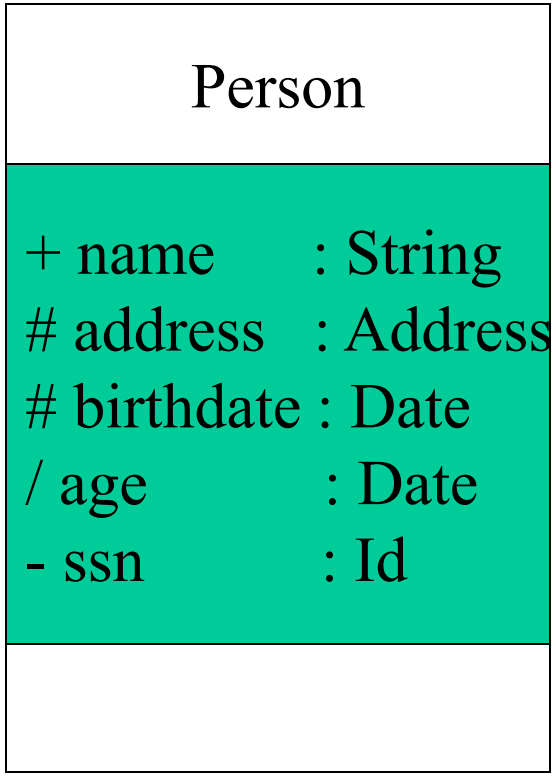
Attributes are usually listed in the form:

attributeName : Type

A *derived* attribute is one that can be computed from other attributes, but doesn't actually exist. For example, a Person's age can be computed from his birth date. A derived attribute is designated by a preceding '/' as in:

/ age : Date

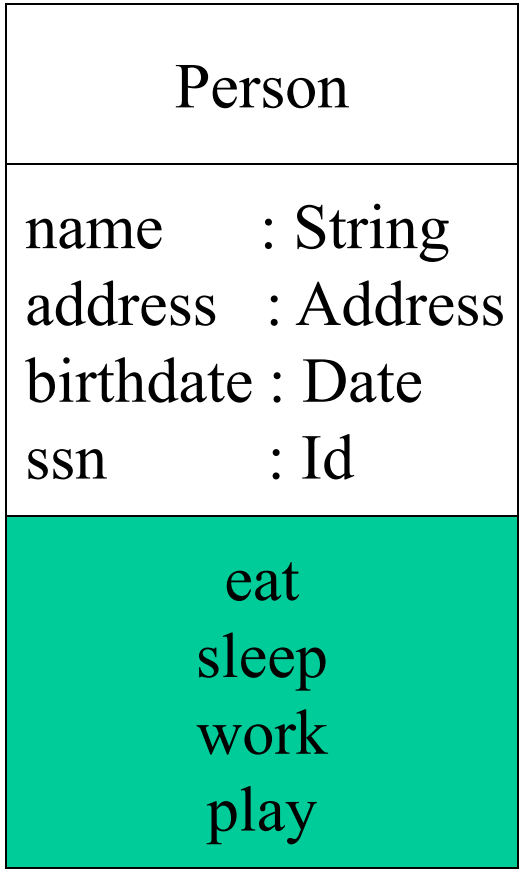
Class Attributes (Cont'd)



Attributes can be:

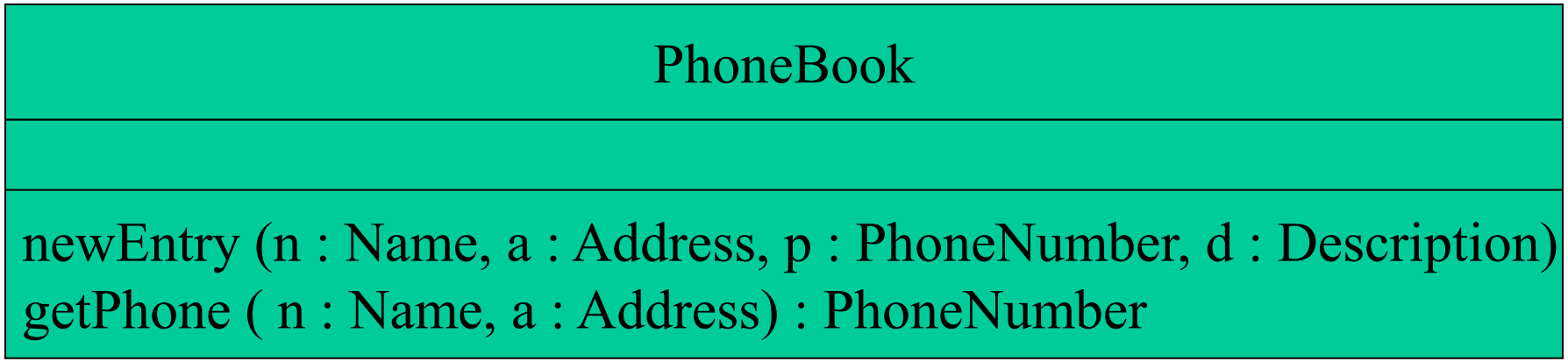
+ public
protected
- private
/ derived

Class Operations



Operations describe the class behavior and appear in the third compartment.

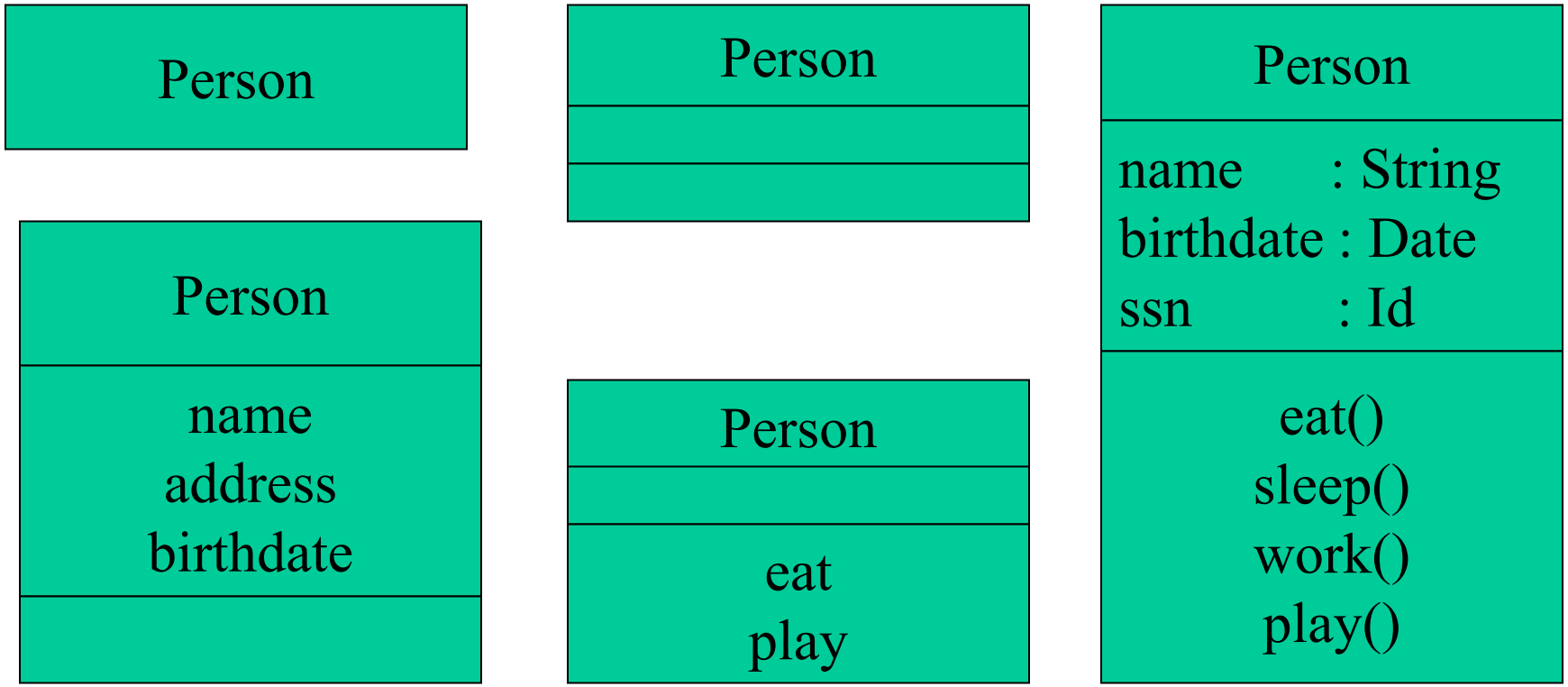
Class Operations (Cont'd)



You can specify an operation by stating its signature: listing the name, type, and default value of all parameters, and, in the case of functions, a return type.

Depicting Classes

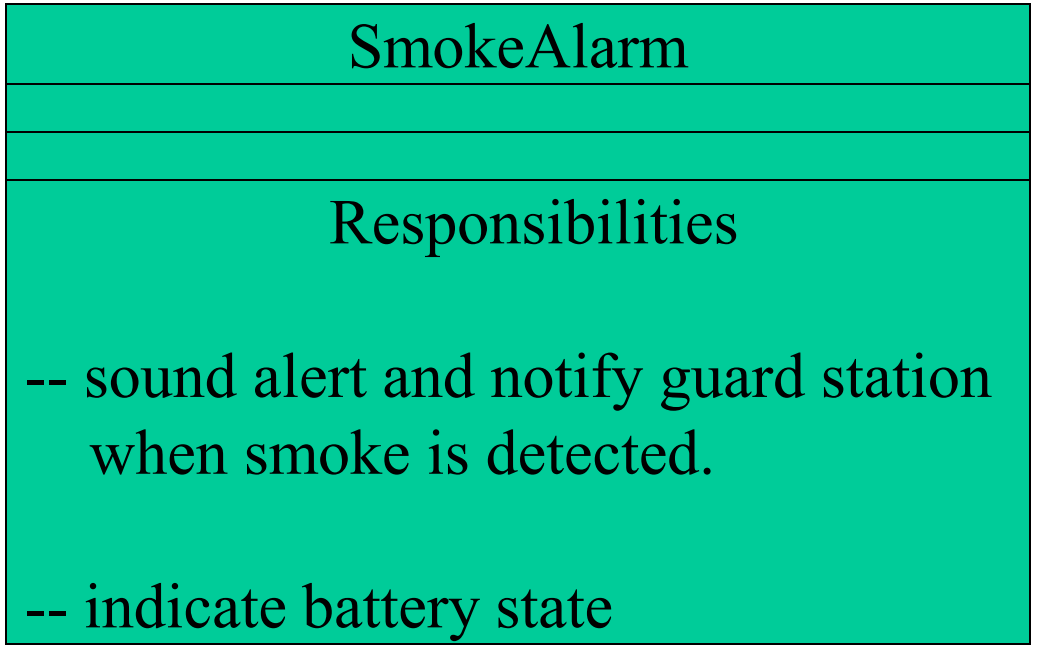
When drawing a class, you needn't show attributes and operation in every diagram.



Class Responsibilities

A class may also include its responsibilities in a class diagram.

A responsibility is a contract or obligation of a class to perform a particular service.



Relationships

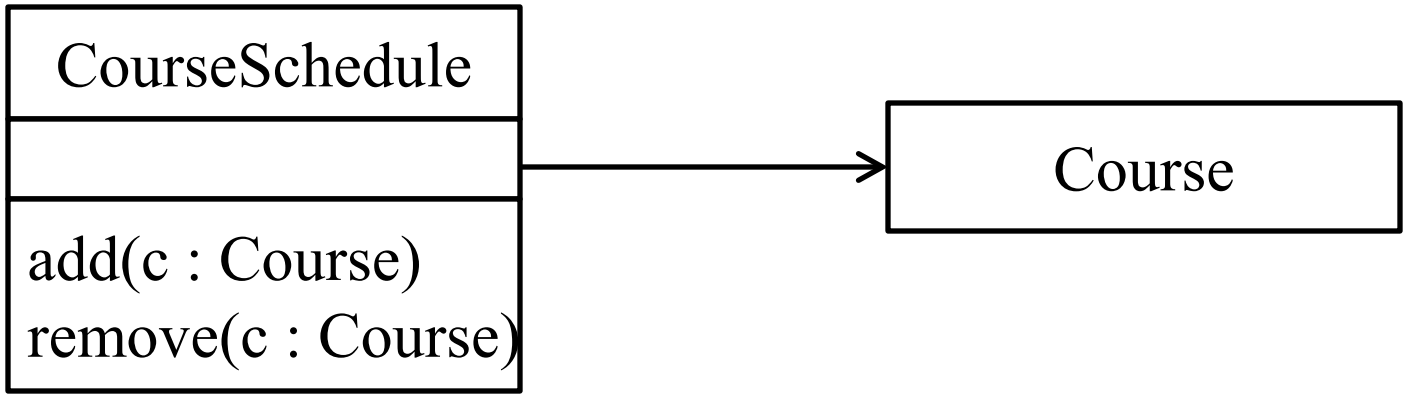
In UML, object interconnections (logical or physical), are modeled as relationships.

There are three kinds of relationships in UML:

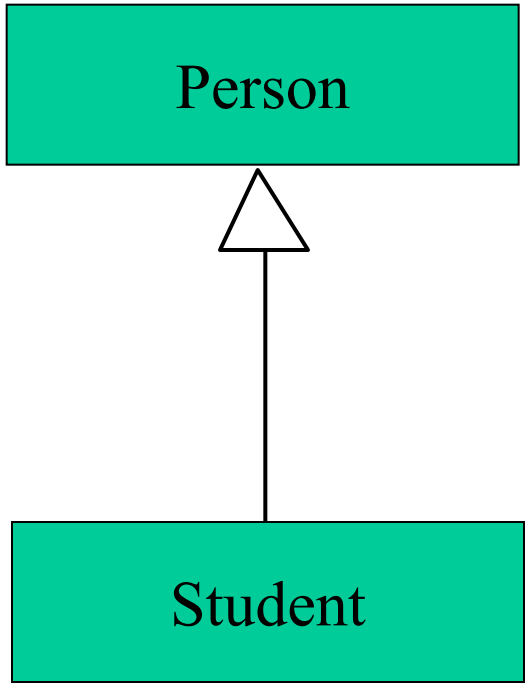
- dependencies
- generalizations
- associations

Dependency Relationships

A *dependency* indicates a semantic relationship between two or more elements. The dependency from *CourseSchedule* to *Course* exists because *Course* is used in both the **add** and **remove** operations of *CourseSchedule*.



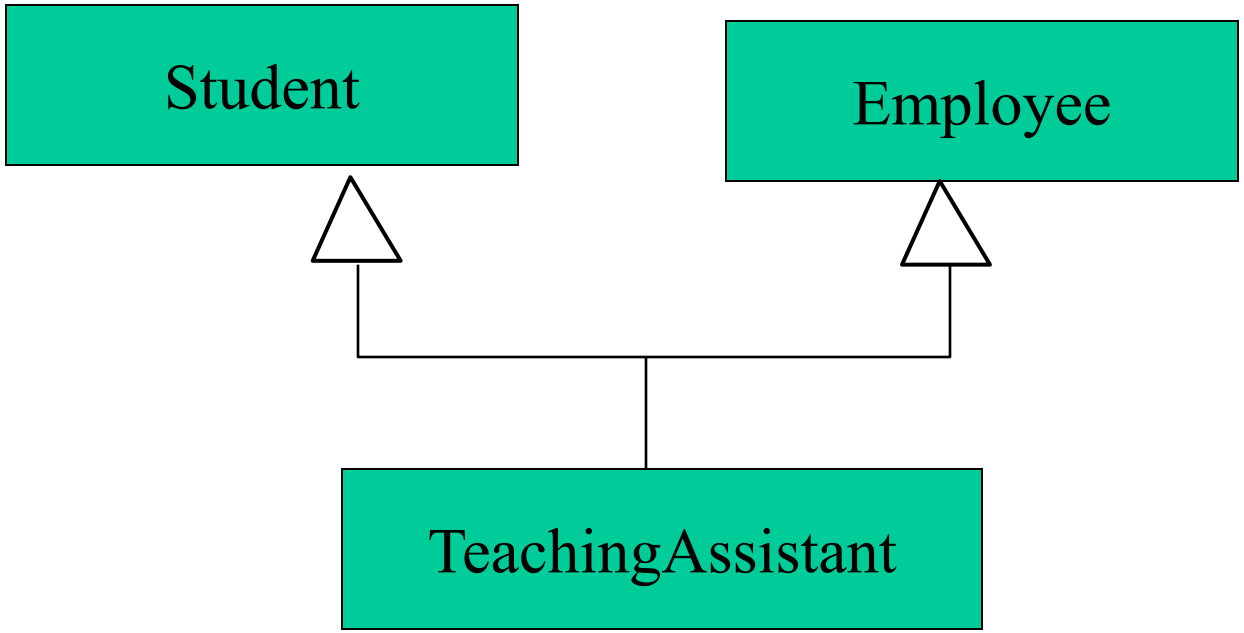
Generalization Relationships



A *generalization* connects a subclass to its superclass. It denotes an inheritance of attributes and behavior from the superclass to the subclass and indicates a specialization in the subclass of the more general superclass.

Generalization Relationships (Cont'd)

UML permits a class to inherit from multiple superclasses.



Association Relationships

If two classes in a model need to communicate with each other, there must be link between them.

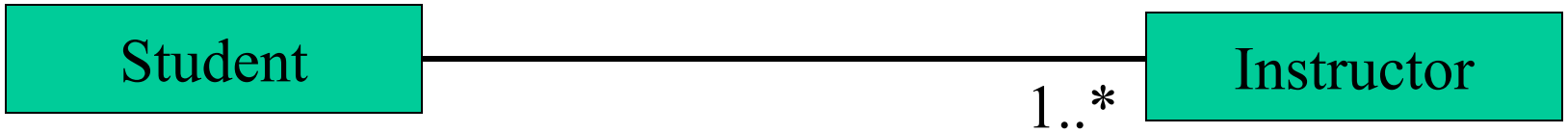
An *association* denotes that link.



Association Relationships (Cont'd)

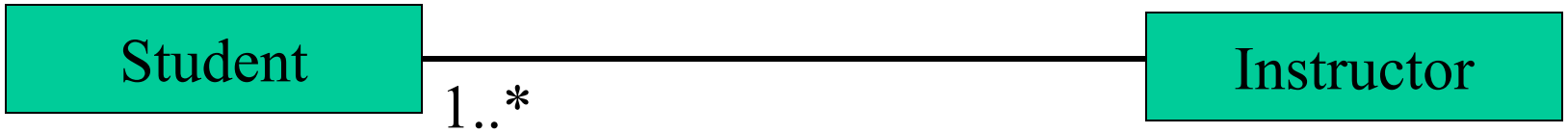
We can indicate the *multiplicity* of an association by adding a *multiplicity adornments* to the line denoting the association.

The example indicates that a *Student* has one or more *Instructors*:



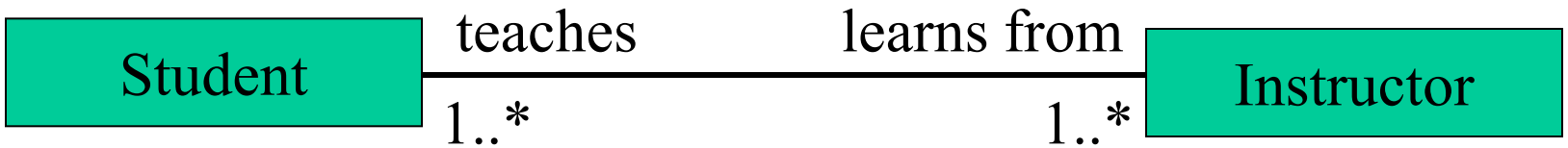
Association Relationships (Cont'd)

The example indicates that every *Instructor* has one or more *Students*:



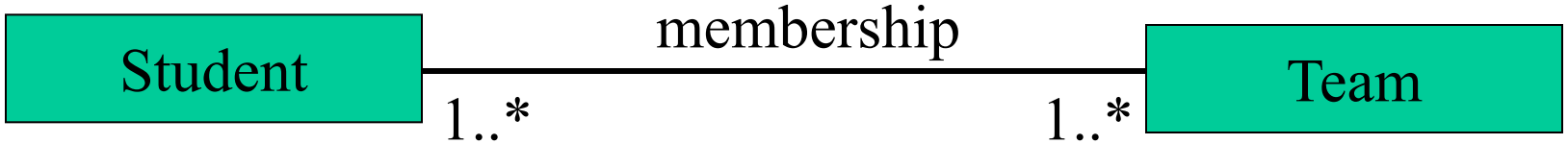
Association Relationships (Cont'd)

We can also indicate the behavior of an object in an association (*i.e.,* the *role* of an object) using *rolenames*.



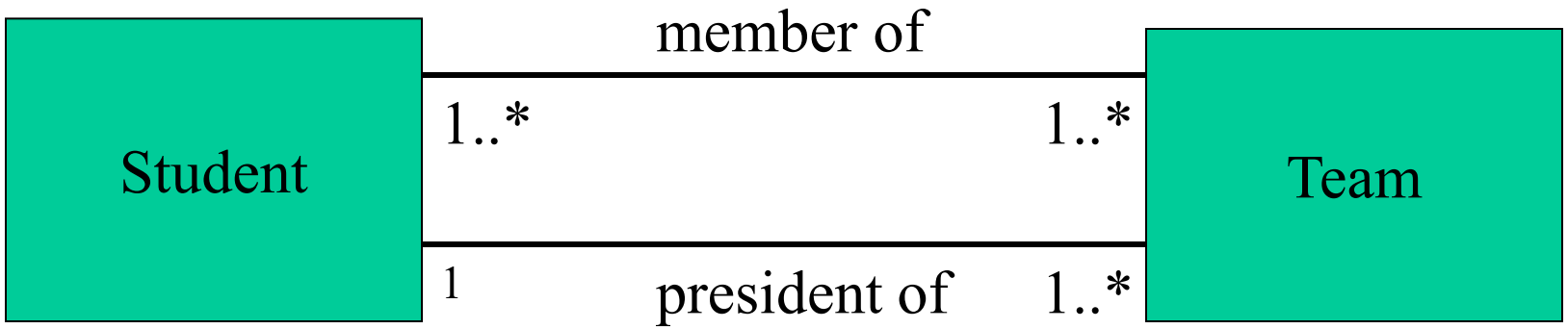
Association Relationships (Cont'd)

We can also name the association.



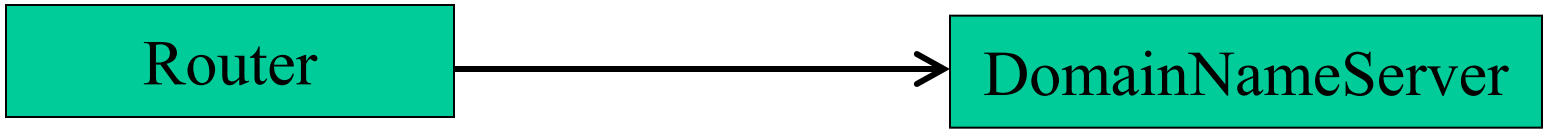
Association Relationships (Cont'd)

We can specify dual associations.



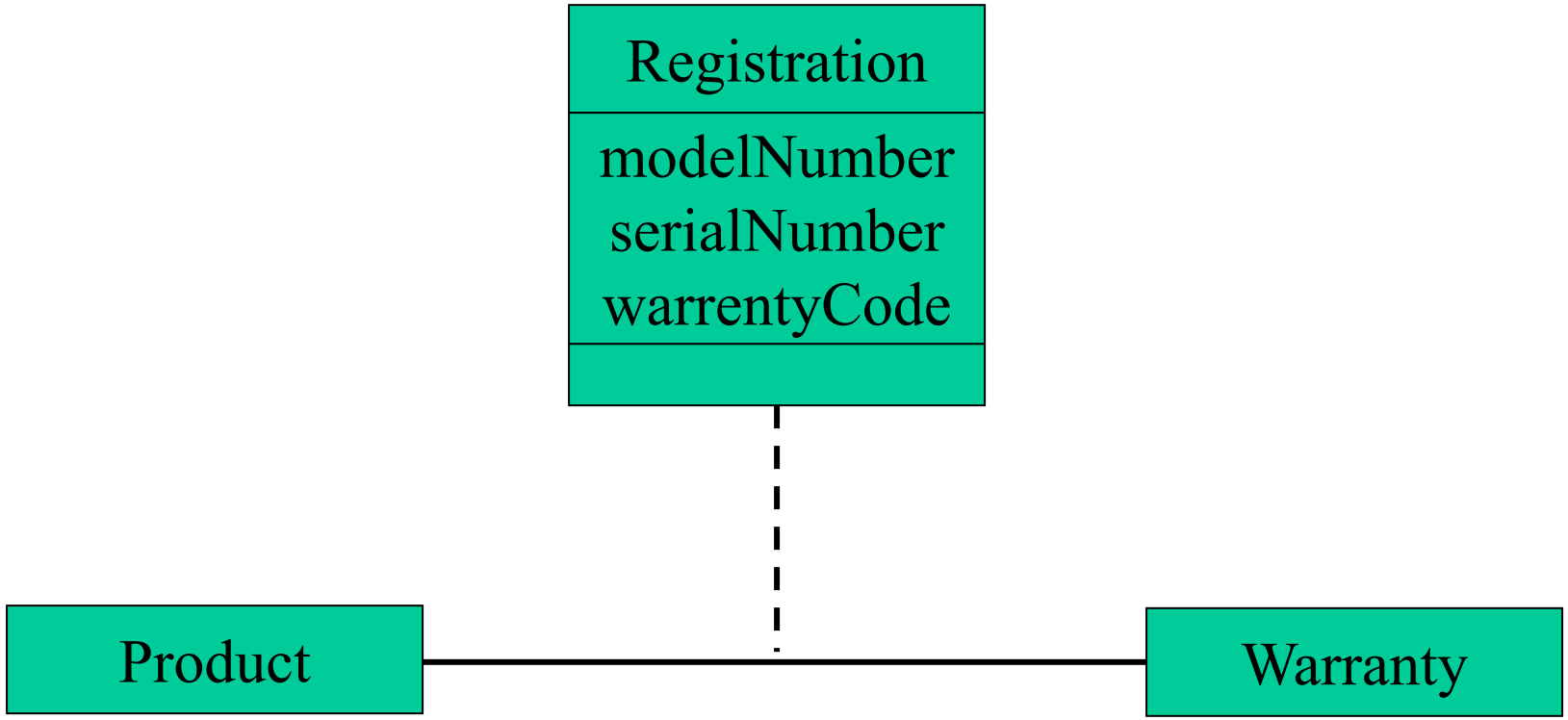
Association Relationships (Cont'd)

We can constrain the association relationship by defining the *navigability* of the association. Here, a *Router* object requests services from a *DNS* object by sending messages to (invoking the operations of) the server. The direction of the association indicates that the server has no knowledge of the *Router*.



Association Relationships (Cont'd)

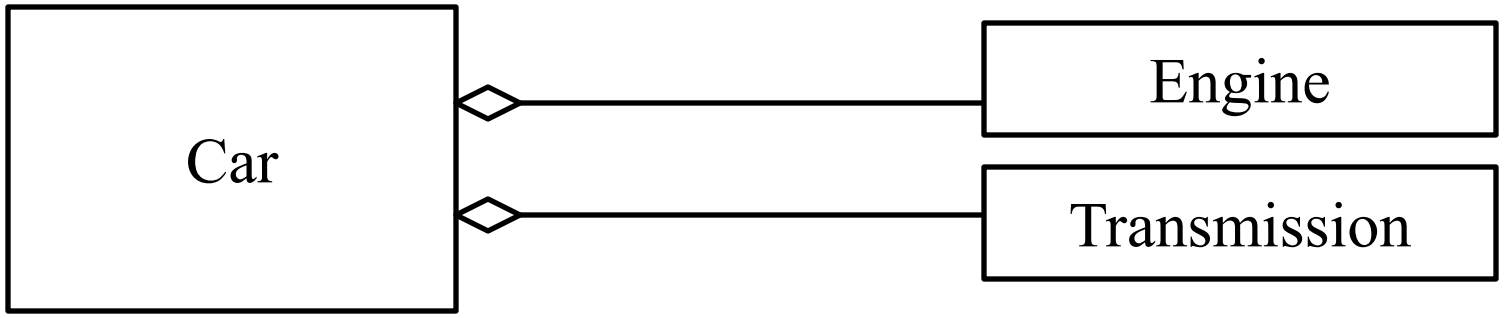
Associations can also be objects themselves, called *link classes* or an *association classes*.



Association Relationships (Cont'd)

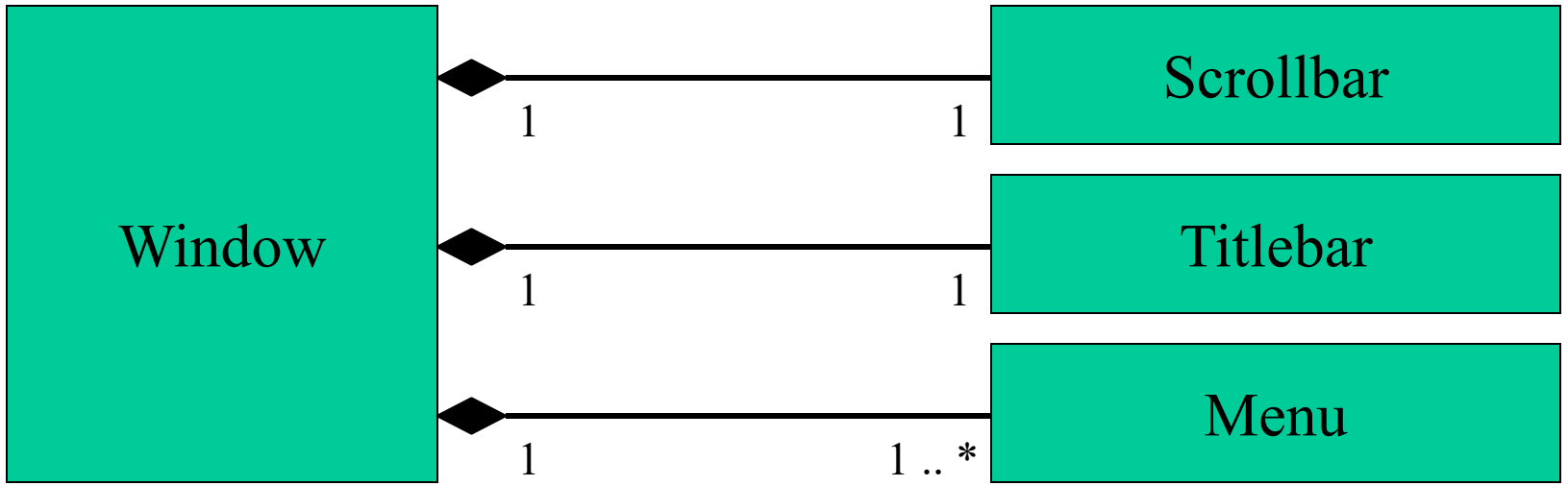
We can model objects that contain other objects by way of special associations called *aggregations* and *compositions*.

An *aggregation* specifies a whole-part relationship between an aggregate (a whole) and a constituent part, where the part can exist independently from the aggregate. Aggregations are denoted by a hollow-diamond adornment on the association.



Association Relationships (Cont'd)

A *composition* indicates a strong ownership and coincident lifetime of parts by the whole (*i.e.*, they live and die as a whole). Compositions are denoted by a filled-diamond adornment on the association.



https://www.digitalocean.com/community/tutorials/composition-in-java-example

Interfaces

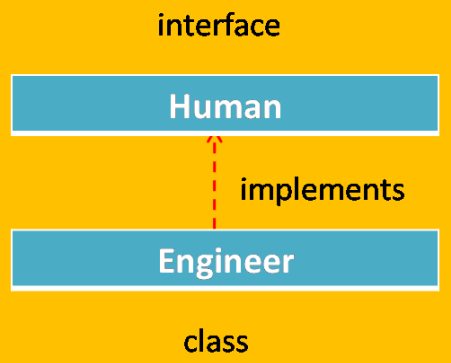


An *interface* is a named set of operations that specifies the behavior of objects without showing their inner structure. It can be rendered in the model by a one- or two-compartment rectangle, with the *stereotype* <<interface>> above the interface name.

Example : Java Interface

```
public interface Human
{
        int a=20;
        public abstract void walk();
        public abstract void eat();
}
 class Engineer implements Human
{
        @Override
        public void eat()
        {
        System.out.println("Engineer can eat ");
         }
        @Override
        public void walk()
        {
              System.out.println("Engineer can walks");
        }
        public static void main(String[] args)
        {
               Engineer E=new Engineer();
               E.eat();
               E.walk();
        }
}
```

abstract method declaration

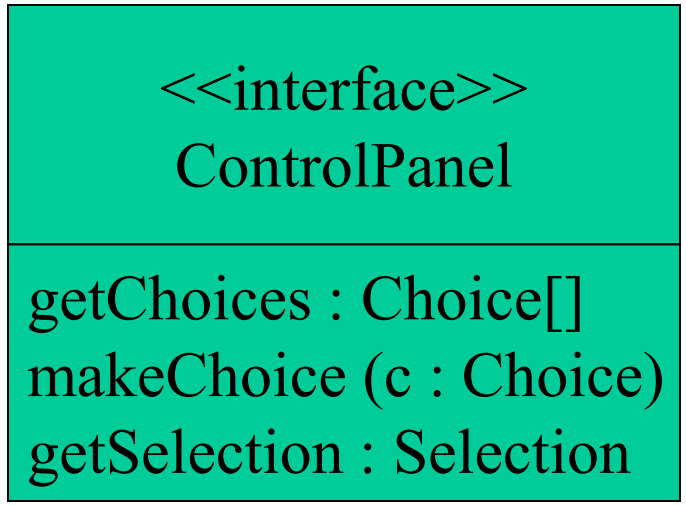


Since Engineer class implements Human interface overridding of abstract methods is mandatory

OUTPUT:

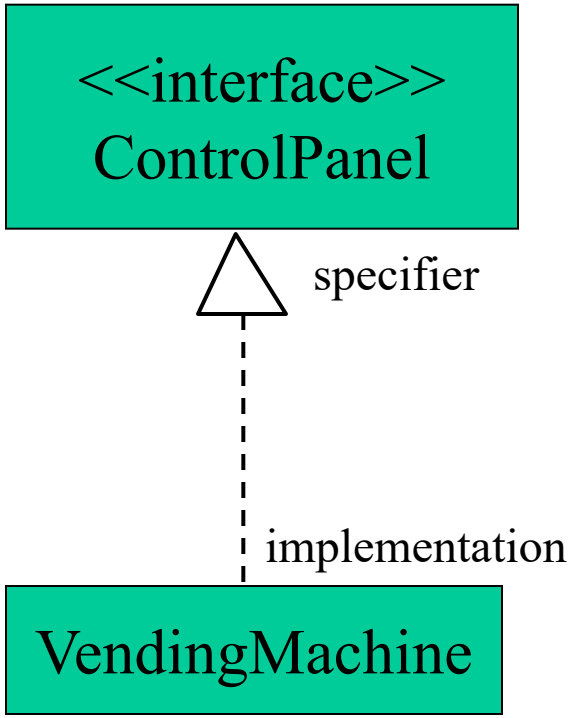
Engineer can eat
Engineer can walks

Interface Services



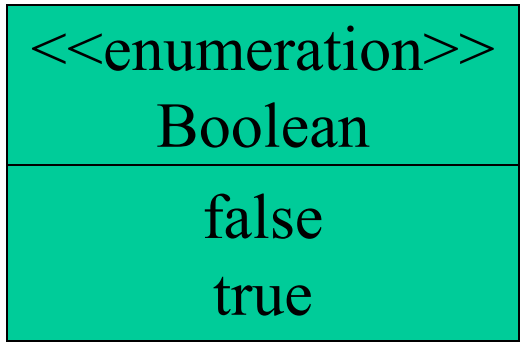
Interfaces do not get instantiated.
They have no attributes or state.
Rather, they specify the services offered by a related class.

Interface Realization Relationship

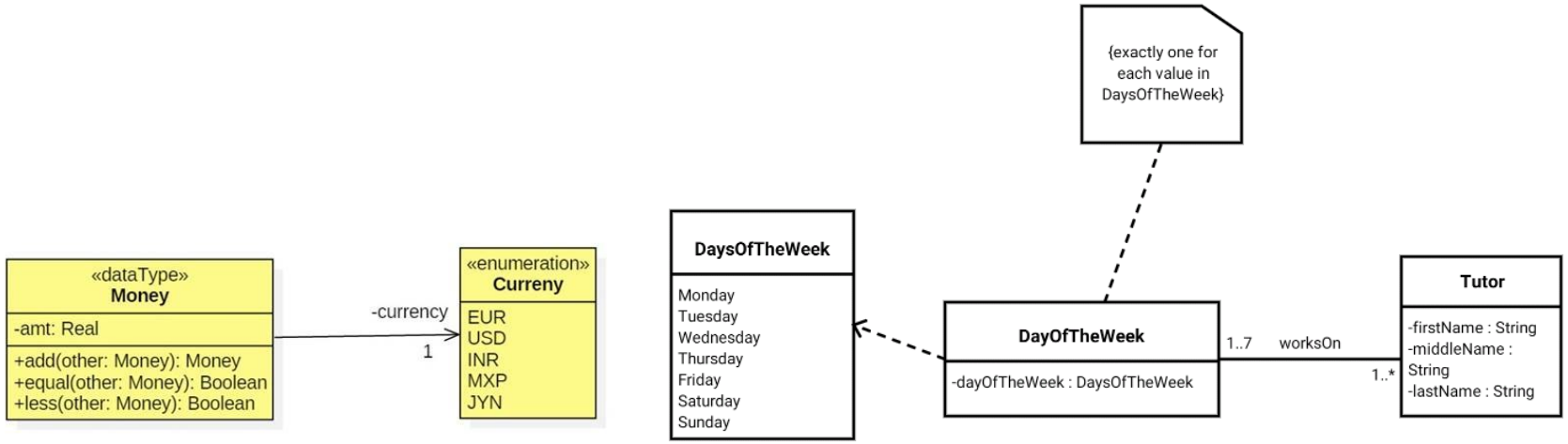


A *realization* relationship connects a class with an interface that supplies its behavioral specification. It is rendered by a dashed line with a hollow triangle towards the specifier.

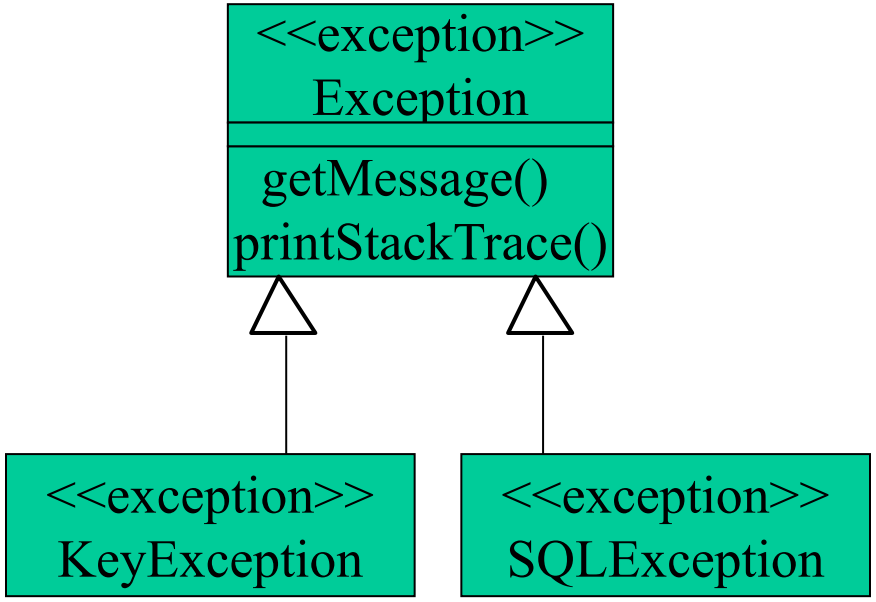
Enumeration



An *enumeration* is a user-defined data type that consists of a name and an ordered list of enumeration literals.



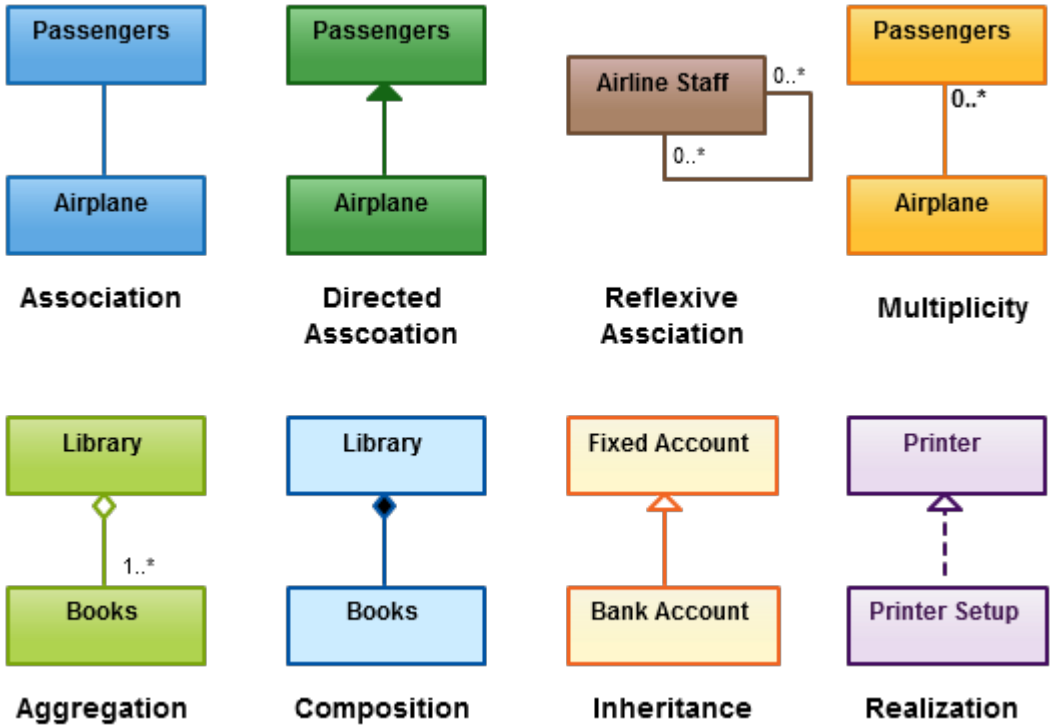
Exceptions

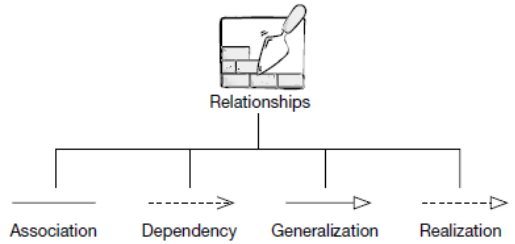


Exceptions can be modeled just like any other class.

Notice the <<exception>> stereotype in the name compartment.

Class Relationships

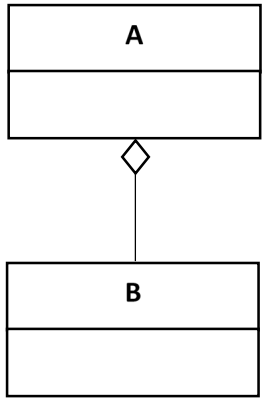




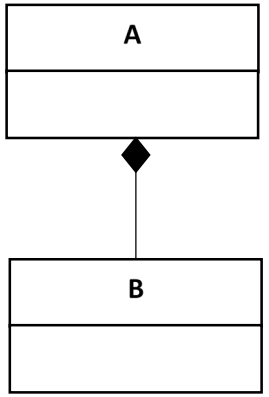
https://www.guru99.com/uml-relationships-with-example.html

Class Relationship (Contd.)

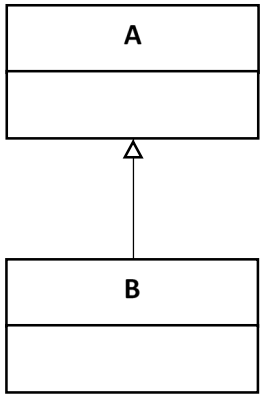
Name the associations.



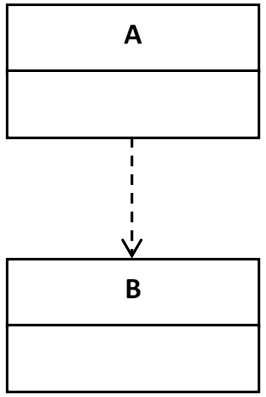
1. _____



2. _____



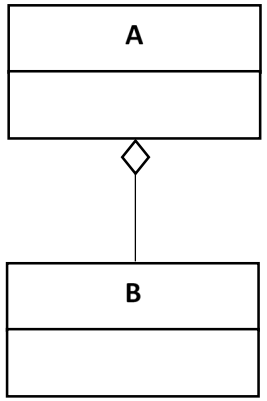
3. _____



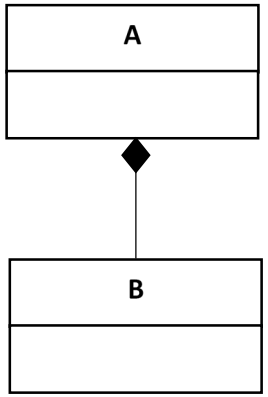
4. _____

Class Relationship (Contd.)

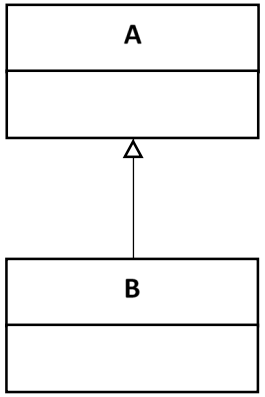
Name the associations.



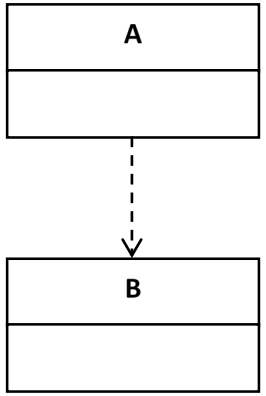
1. A aggregates B, B is part of A



2. A is composed of B; A contains B type objects

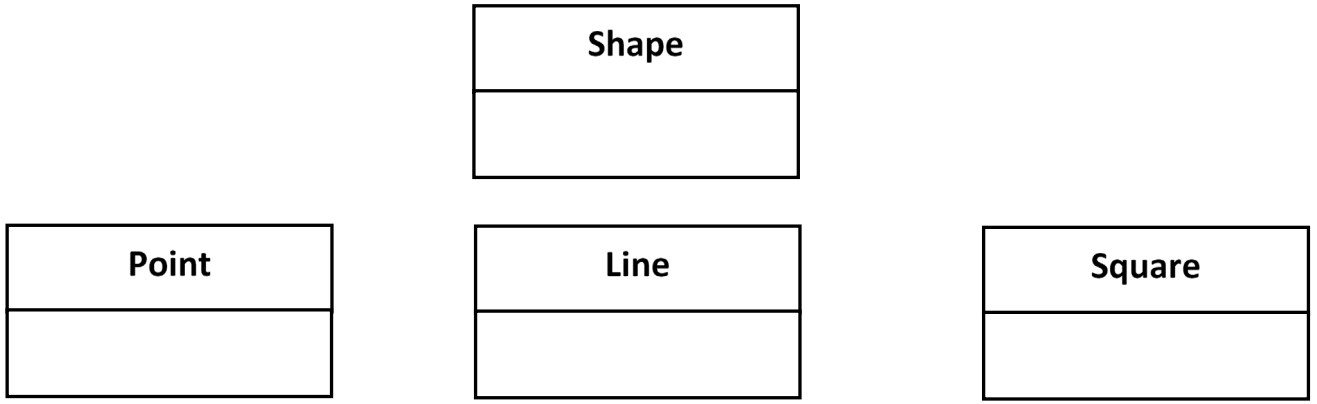


3. B is derived from A, A generalizes B



4. A depends upon B, A uses B

Define Relationships



Exercise

Draw a class diagram for an Airport. The Airport has Aircraft, which is not part of the Airport. But we can still say Airport has them. The Aircraft is either a Jet or a Helicopter.

Sequence Diagram

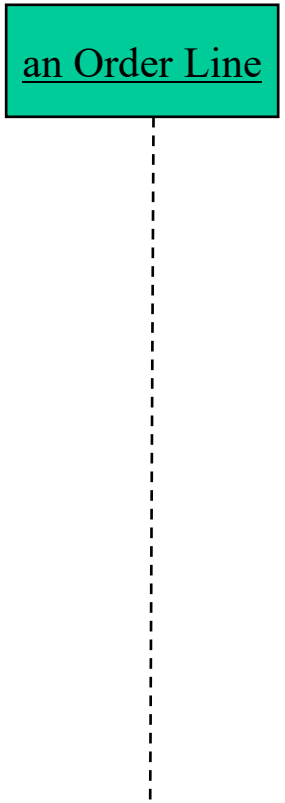
A *sequence diagram* is an interaction diagram that emphasizes the time ordering of messages. It shows a set of objects and the messages sent and received by those objects.

Graphically, a sequence diagram is a table that shows objects arranged along the X axis and messages, ordered in increasing time, along the Y axis.

- *The UML User Guide, [Booch,99]*

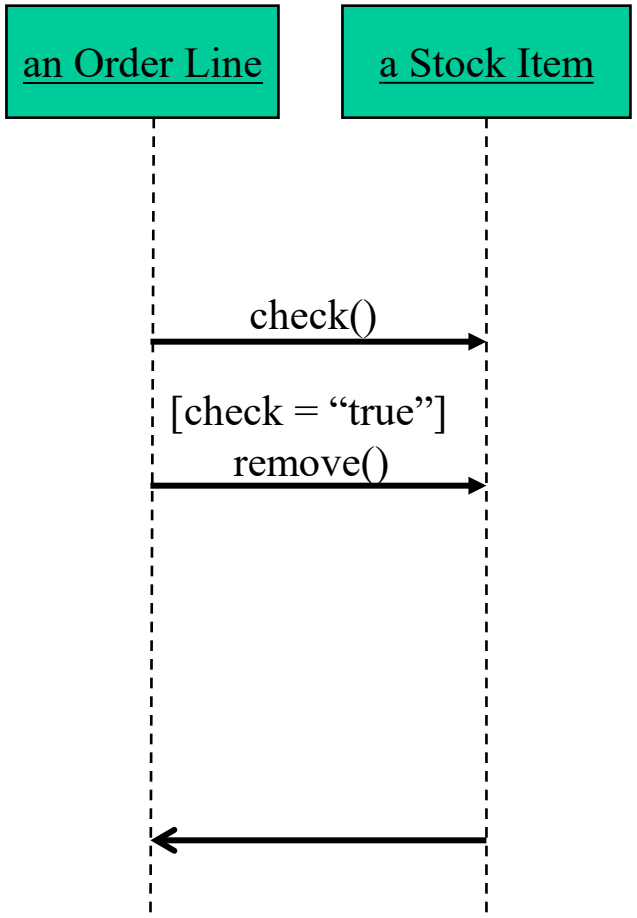
https://creately.com/blog/diagrams/sequence-diagram-tutorial/

Sequence Diagram



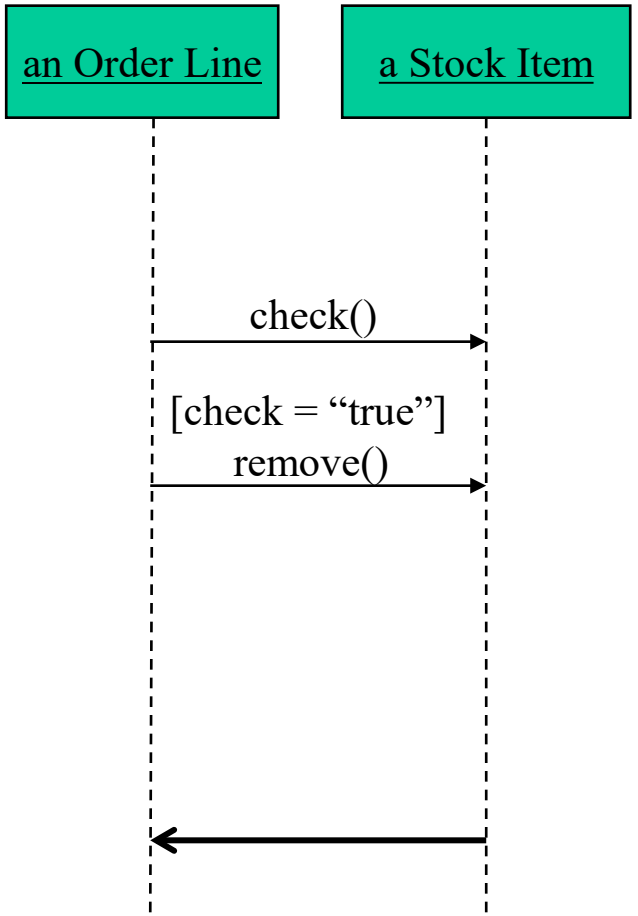
An object in a sequence diagram is rendered as a box with a dashed line descending from it. The line is called the *object lifeline*, and it represents the existence of an object over a period of time.

Sequence Diagram



Messages are rendered as horizontal arrows being passed from object to object as time advances down the object lifelines. Conditions (such as [check = “true”]) indicate when a message gets passed.

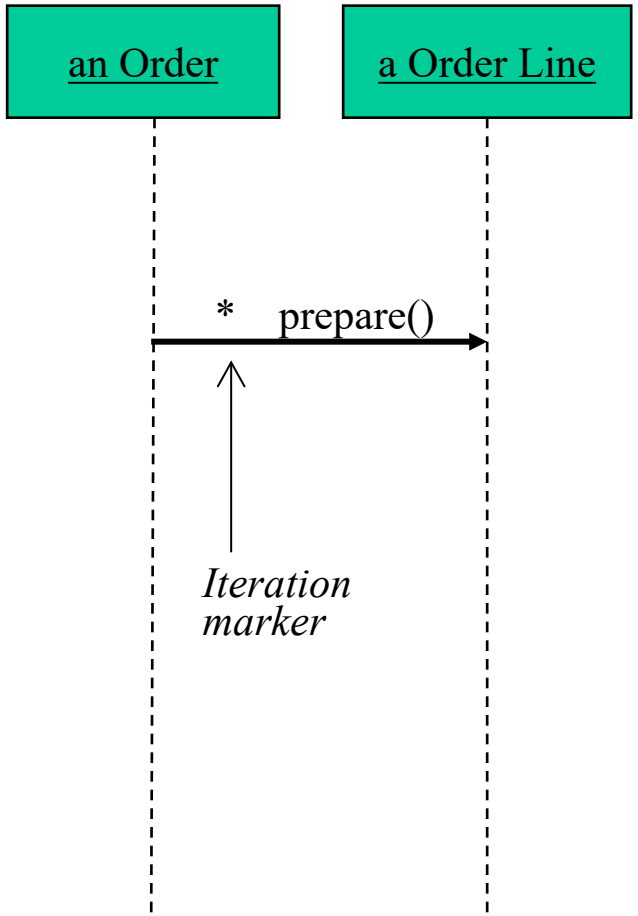
Sequence Diagram



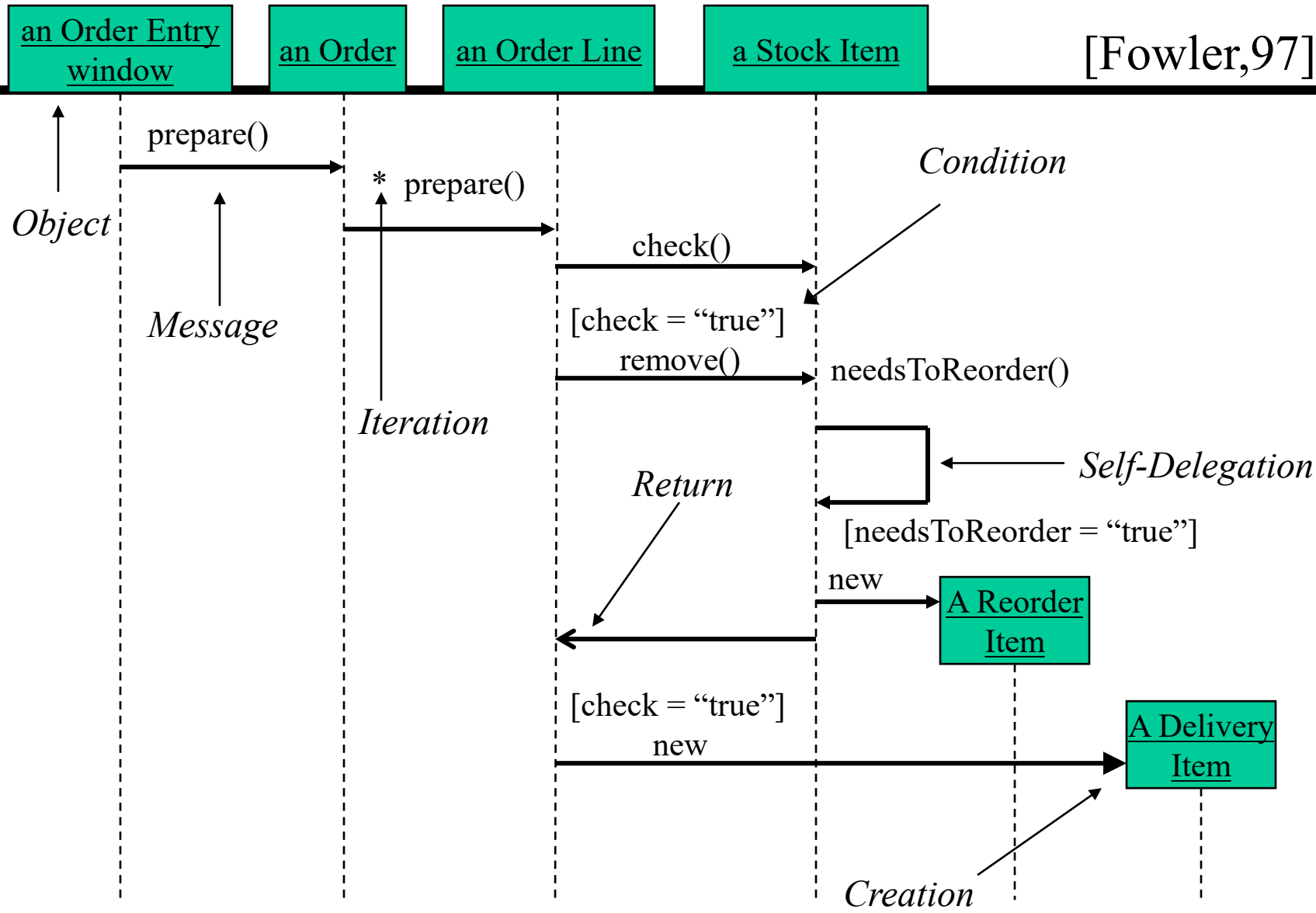
Notice that the bottom arrow is different. The arrow head is not solid, and there is no accompanying message.

This arrow indicates a **return** from a previous message, not a new message.

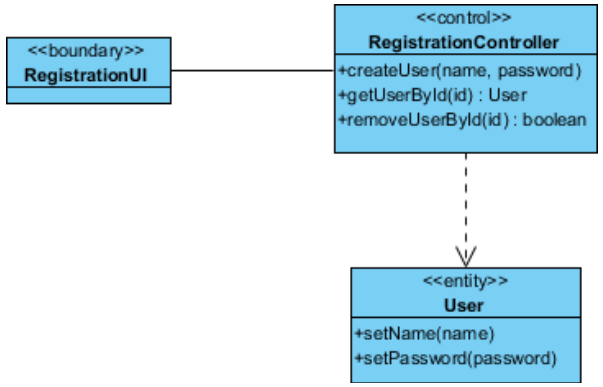
Sequence Diagram

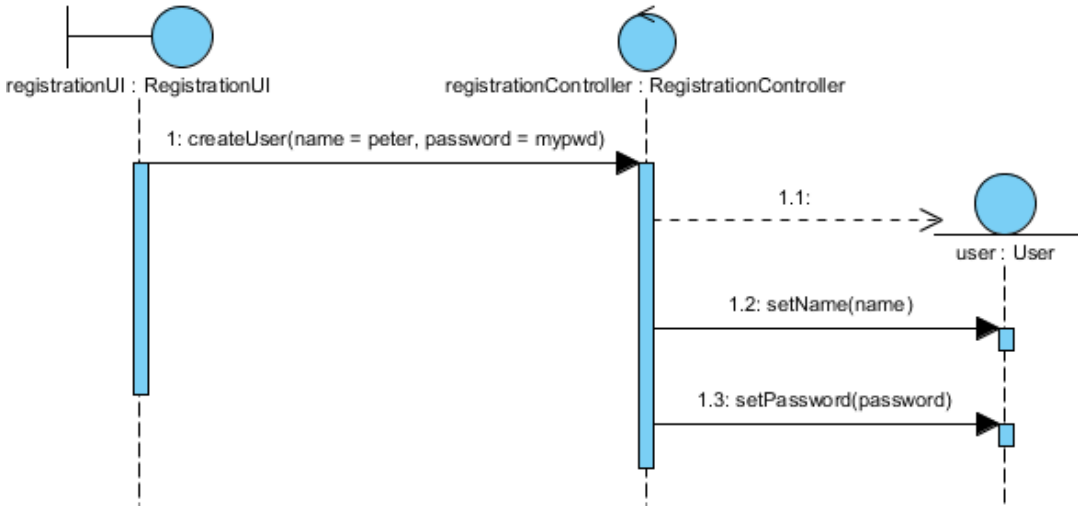


An iteration marker, such as * (as shown), or *[i = 1..n] , indicates that a message will be repeated as indicated.


an Order Entry window
an Order
an Order Line
a Stock Item
[Fowler,97]
Object
prepare()
Message
* prepare()
Iteration
check()
Condition
[check = "true"]
remove()
needsToReorder()
Self-Delegation
Return
[needsToReorder = "true"]
new
A Reorder Item
[check = "true"]
new
A Delivery Item
Creation

Visual Paradigm

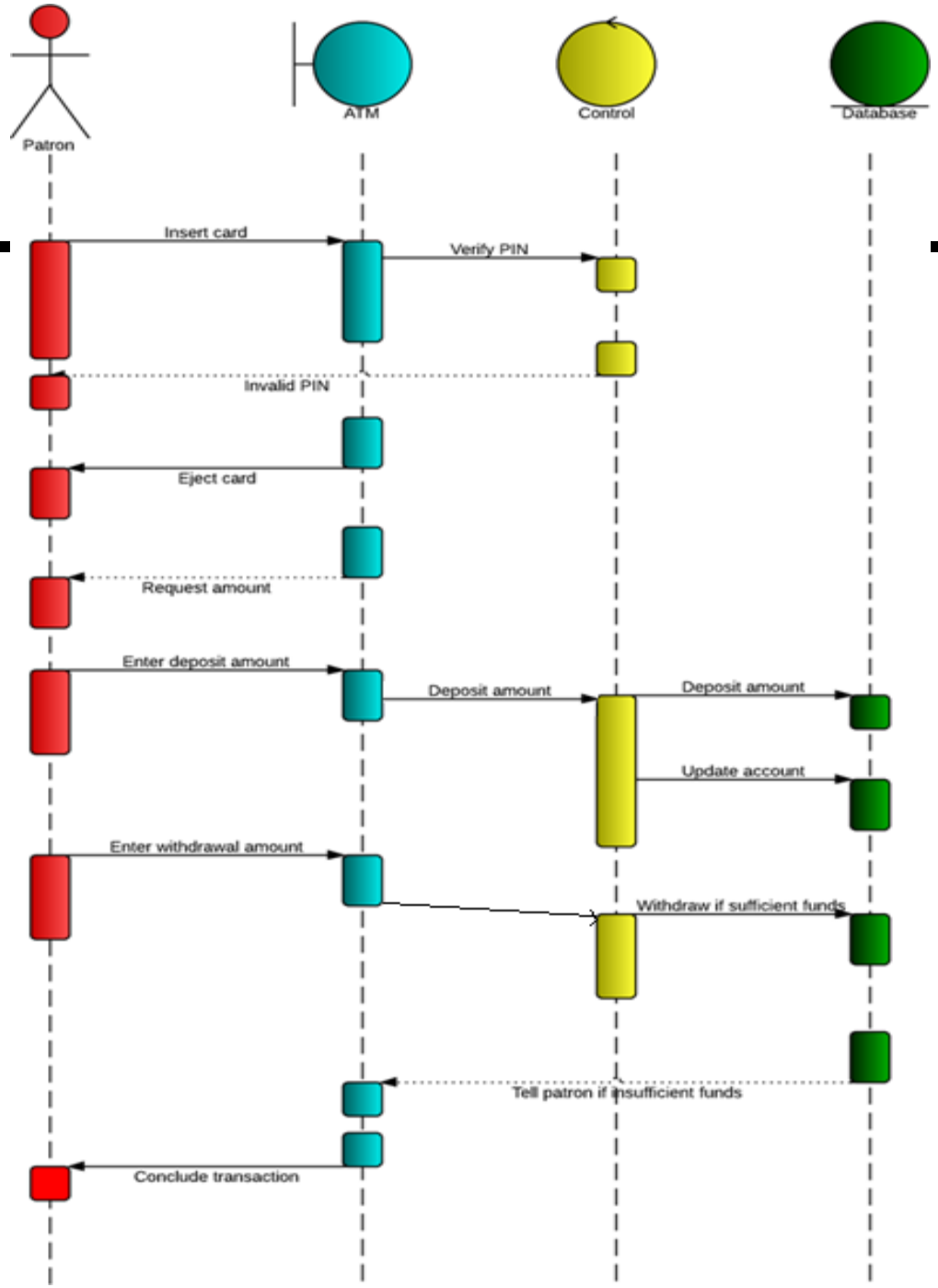




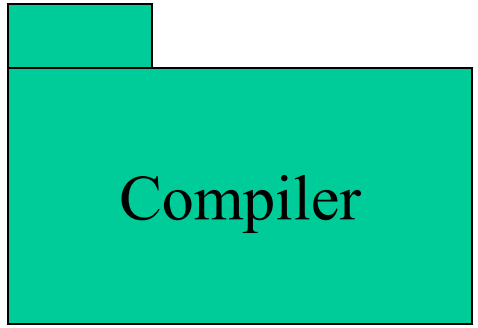
https://www.visual-paradigm.com/tutorials/how-to-draw-uml-sequence-diagram.jsp

Exercise

Draw a sequence diagram for ATM operations where the user can deposit and withdraw cash


Patron
ATM
Control
Database
Insert card
Verify PIN
Invalid PIN
Eject card
Request amount
Enter deposit amount
Deposit amount
Deposit amount
Update account
Enter withdrawal amount
Withdraw if sufficient funds
Tell patron if insufficient funds
Conclude transaction

Packages



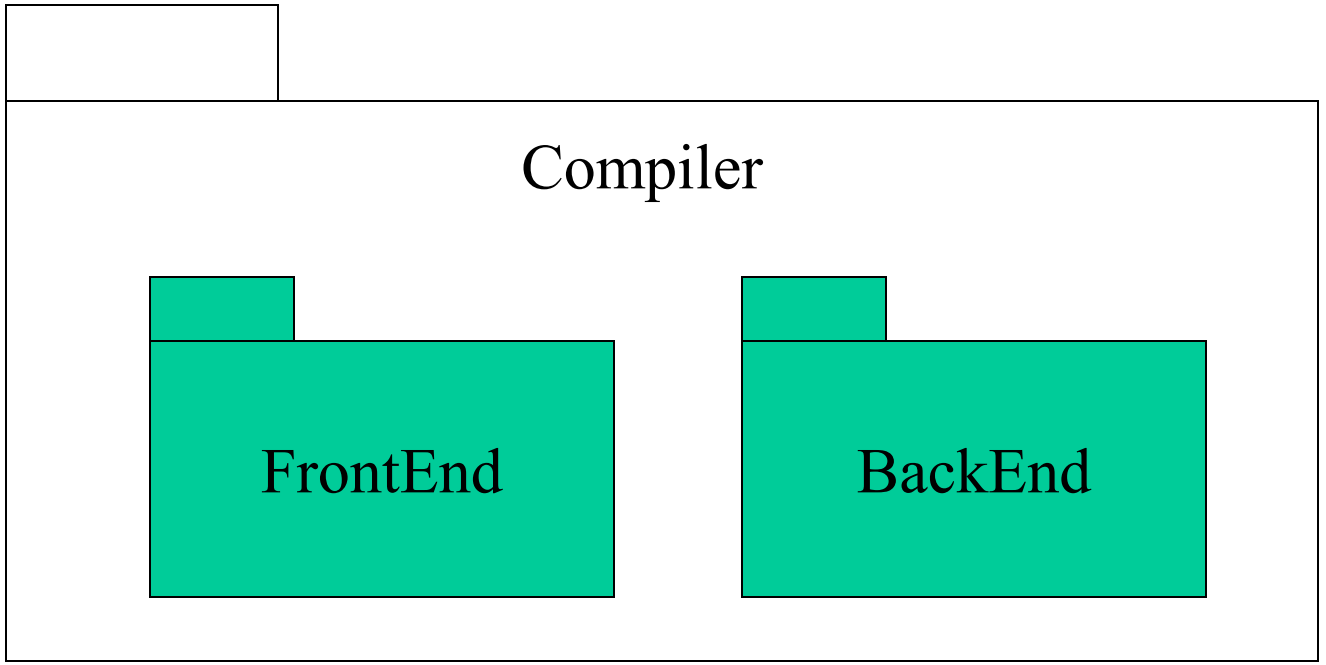
A *package* is a container-like element for organizing other elements into groups.

A package can contain classes and other packages and diagrams.

Packages can be used to provide controlled access between classes in different packages.

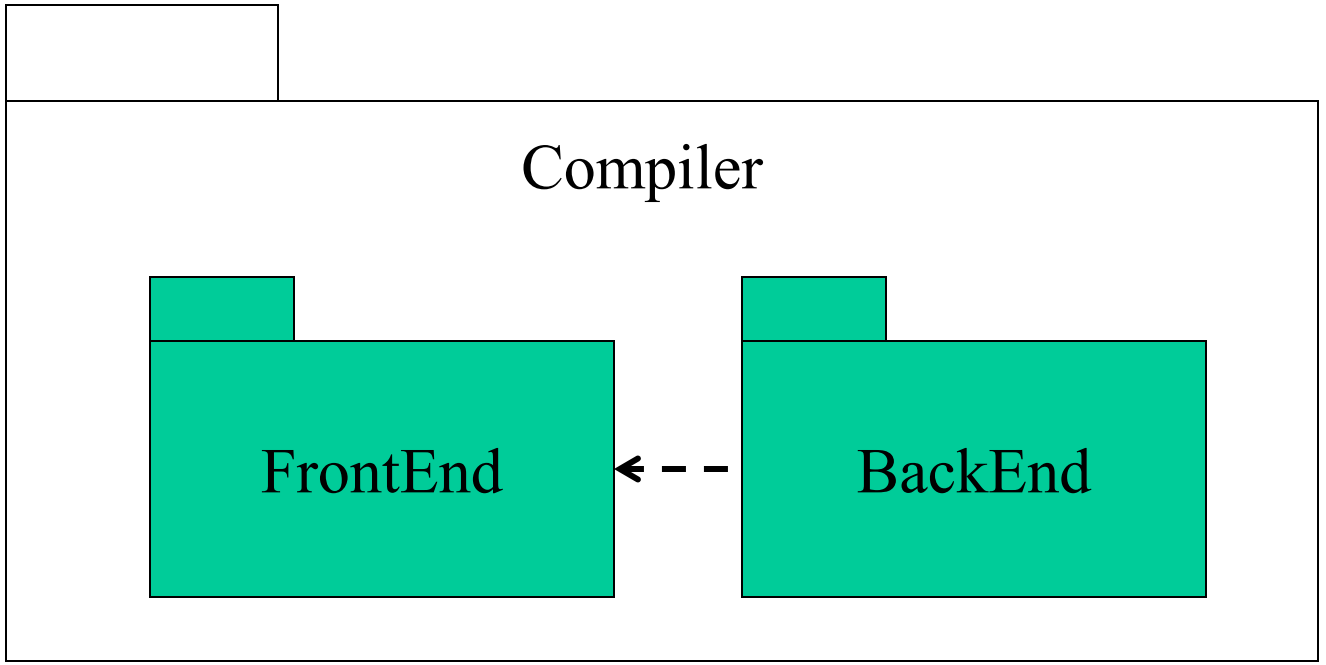
Packages (Cont'd)

Classes in the *FrontEnd* package and classes in the *BackEnd* package cannot access each other in this diagram.

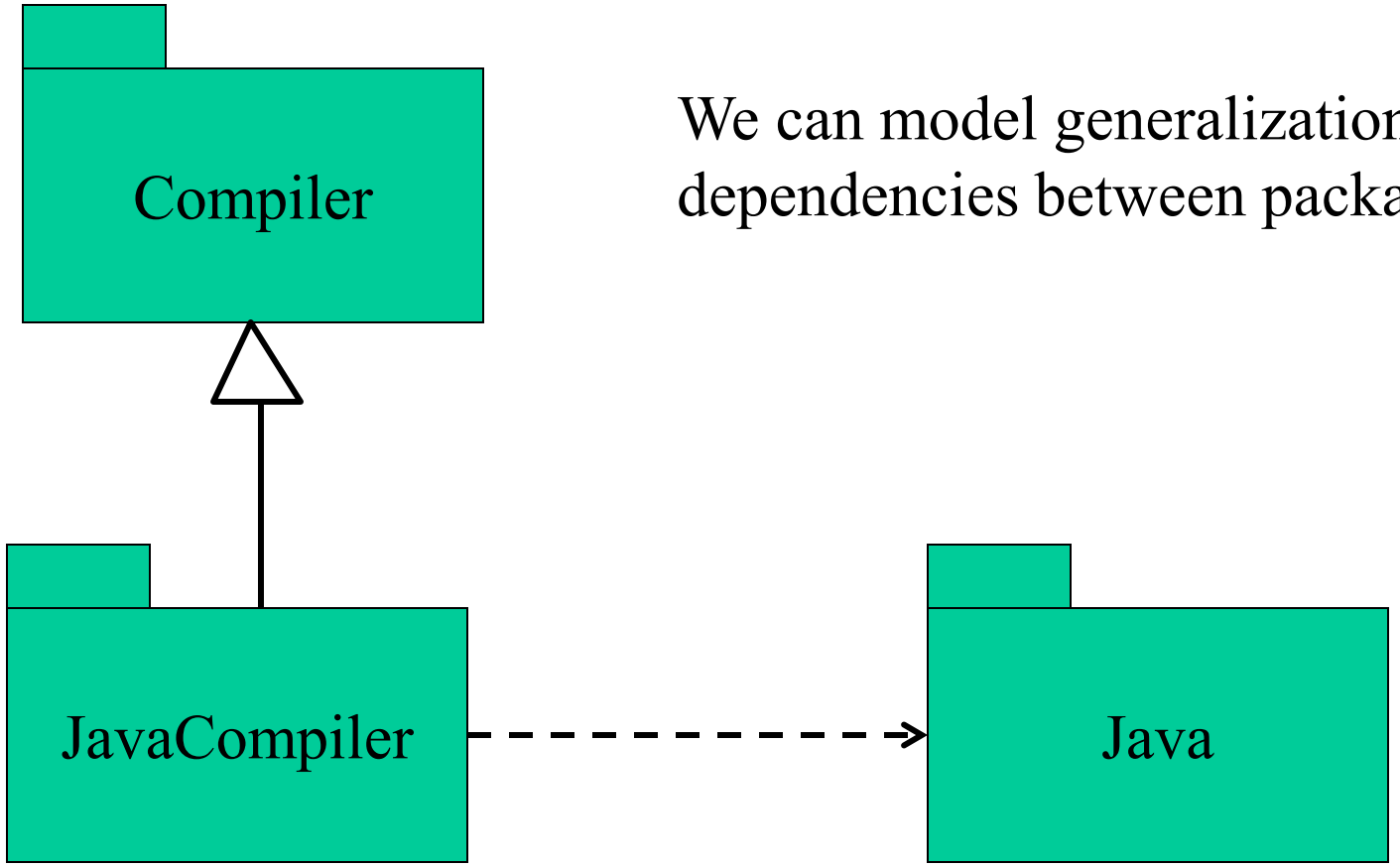


Packages (Cont'd)

Classes in the *BackEnd* package now have access to the classes in the *FrontEnd* package.



Packages (Cont'd)



We can model generalizations and dependencies between packages.

Component Diagram

Component diagrams are one of the two kinds of diagrams found in modeling the physical aspects of an object-oriented system. They show the organization and dependencies between a set of components.

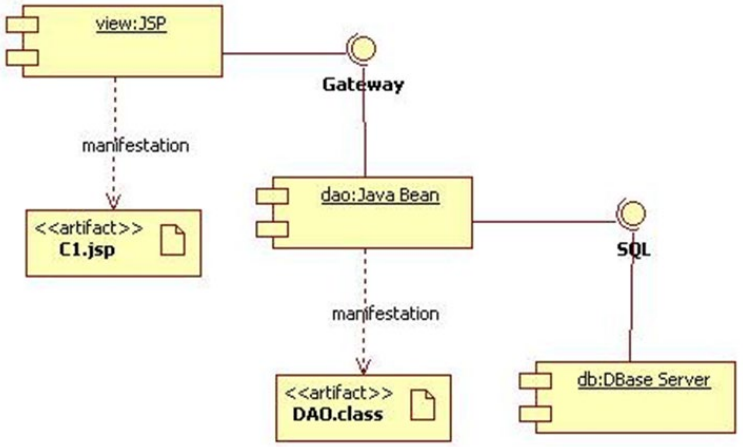
Use component diagrams to model the ***static implementation view*** of a system. This involves modeling the physical things that reside on a node, such as executables, libraries, tables, files, and documents.

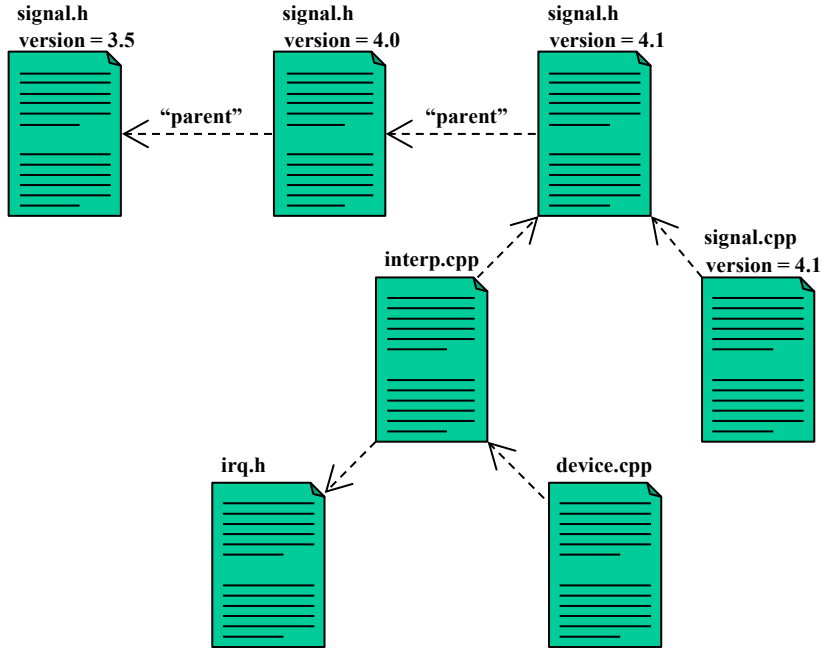
- *The UML User Guide, Booch et. al., 1999*

Component Diagram

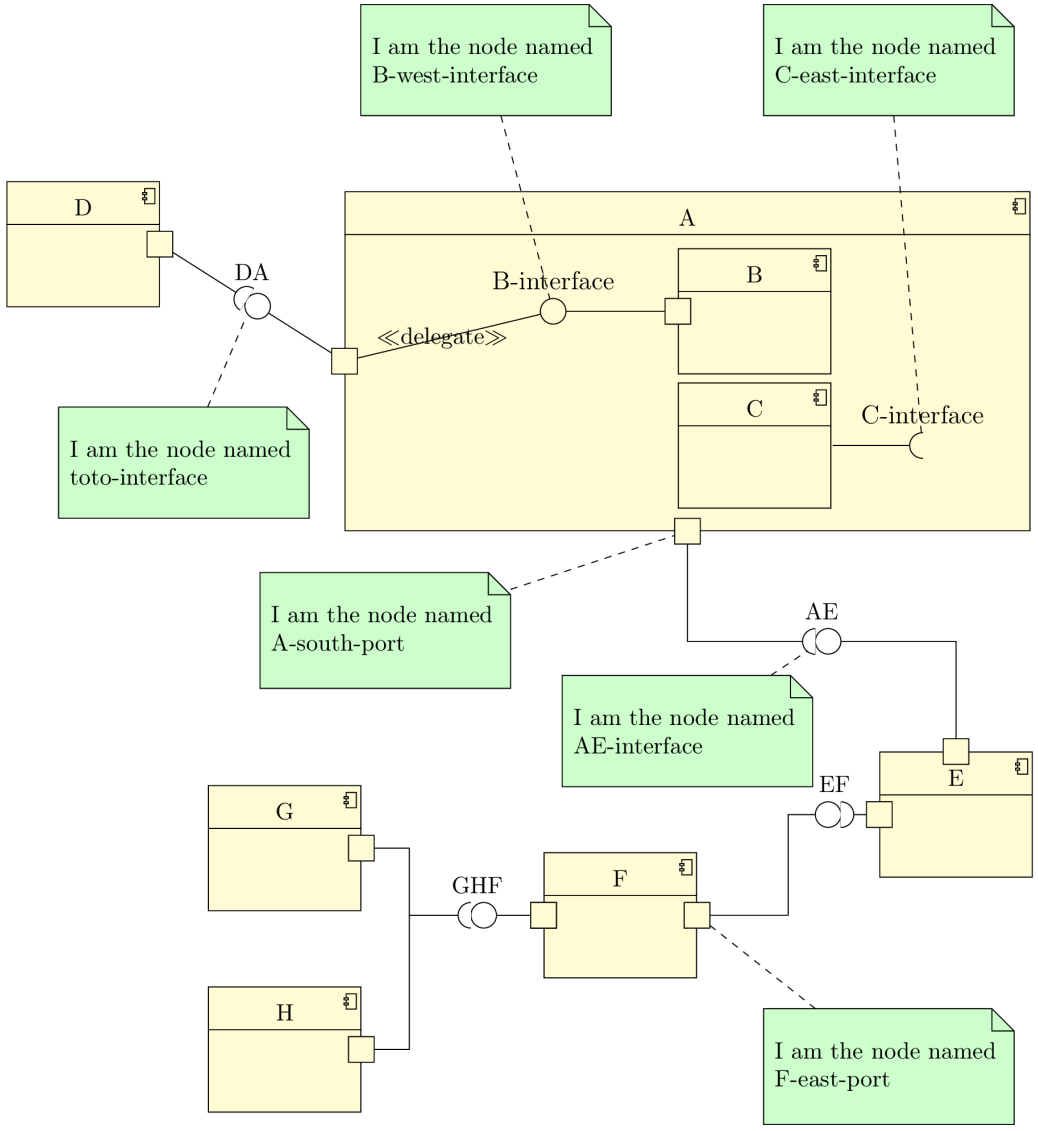
Modeling source code.

[Booch, 99]





Component Diagram



Deployment Diagram

Deployment diagrams are one of the two kinds of diagrams found in modeling the physical aspects of an object-oriented system. They show the configuration of ***run-time processing*** nodes and the components that live on them.

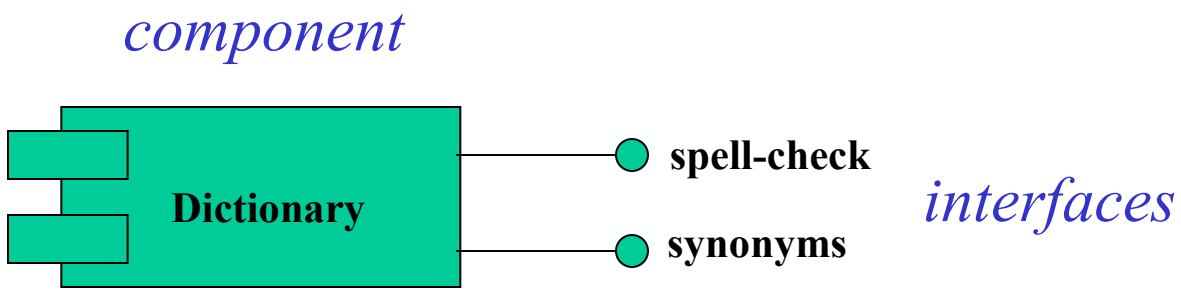
Use deployment diagrams to model the ***static deployment view*** of a system. This involves modeling the topology of the hardware on which the system executes.

- *The UML User Guide, [Booch,99]*

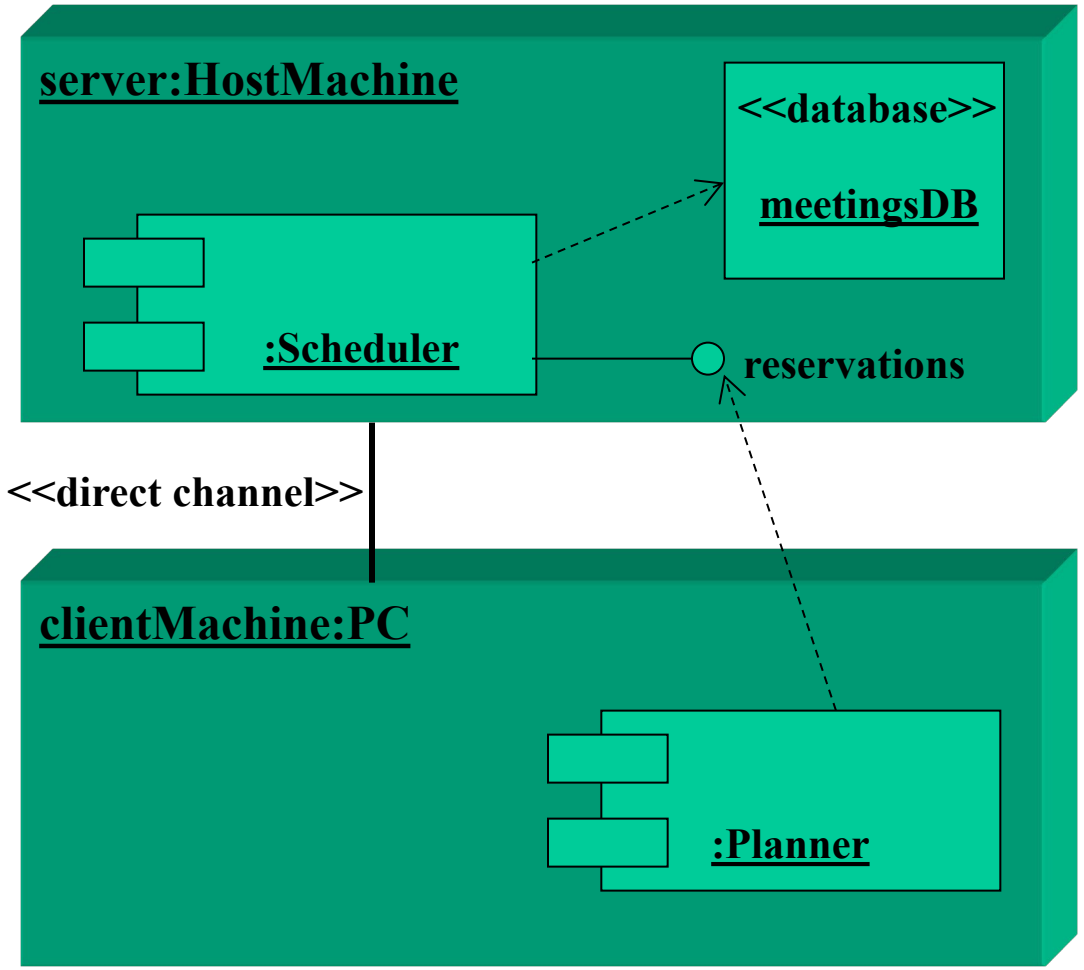
Deployment Diagram

A component is a physical unit of implementation with well-defined interfaces that is intended to be used as a replaceable part of a system. Well designed components do not depend directly on other components, but rather on interfaces that components support.

- *The UML Reference Manual, [Rumbaugh,99]*



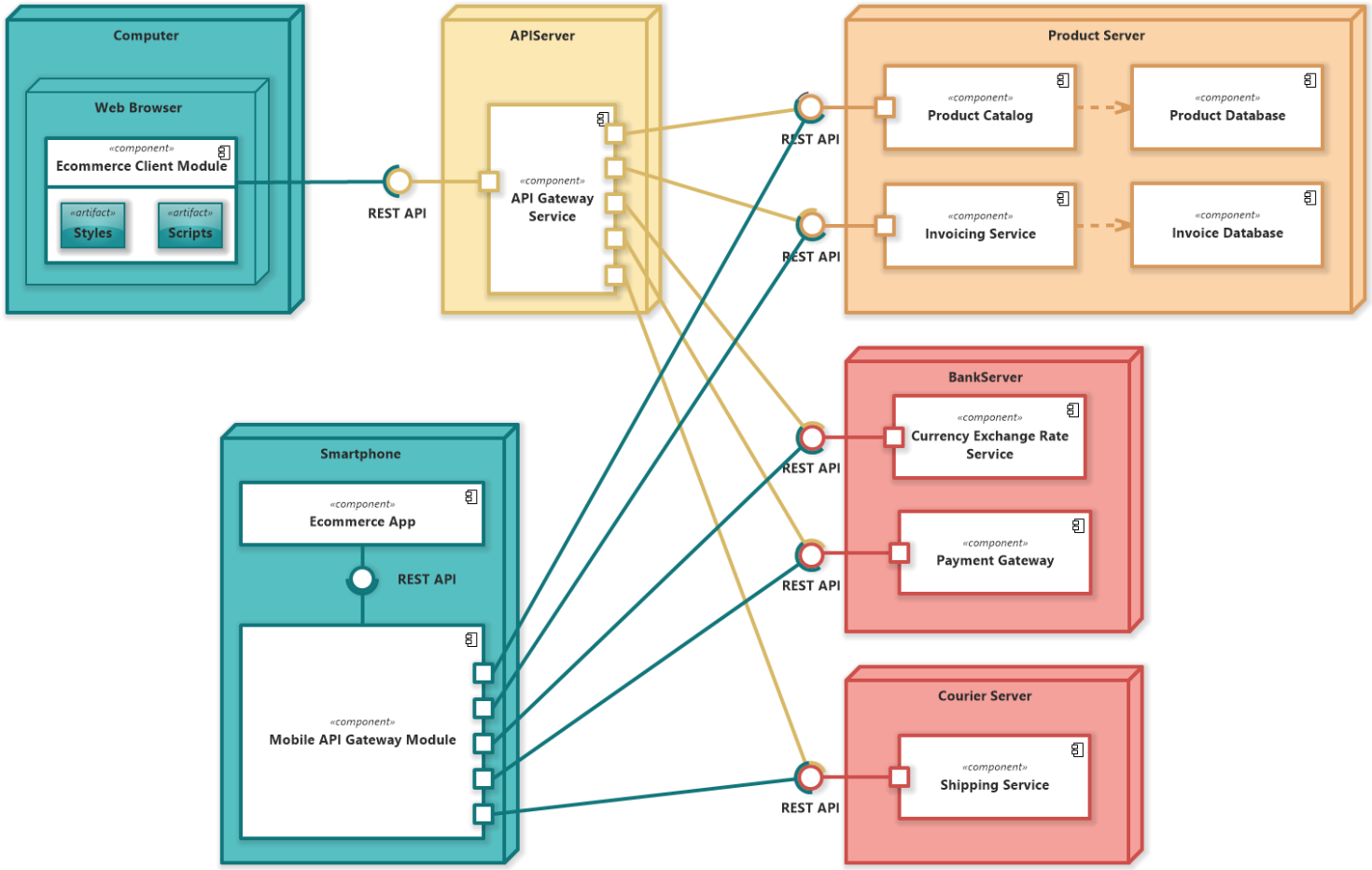
Deployment Diagram



Deployment diagram of a client-server system.

[Rumbaugh,99]

Deployment Diagram



State Machine

"The state machine view describes the dynamic behavior of objects over time by modeling the lifecycles of objects of each class. Each object is treated as an isolated entity that communicates with the rest of the world by detecting events and responding to them. Events represent the kinds of changes that objects can detect... Anything that can affect an object can be characterized as an event."

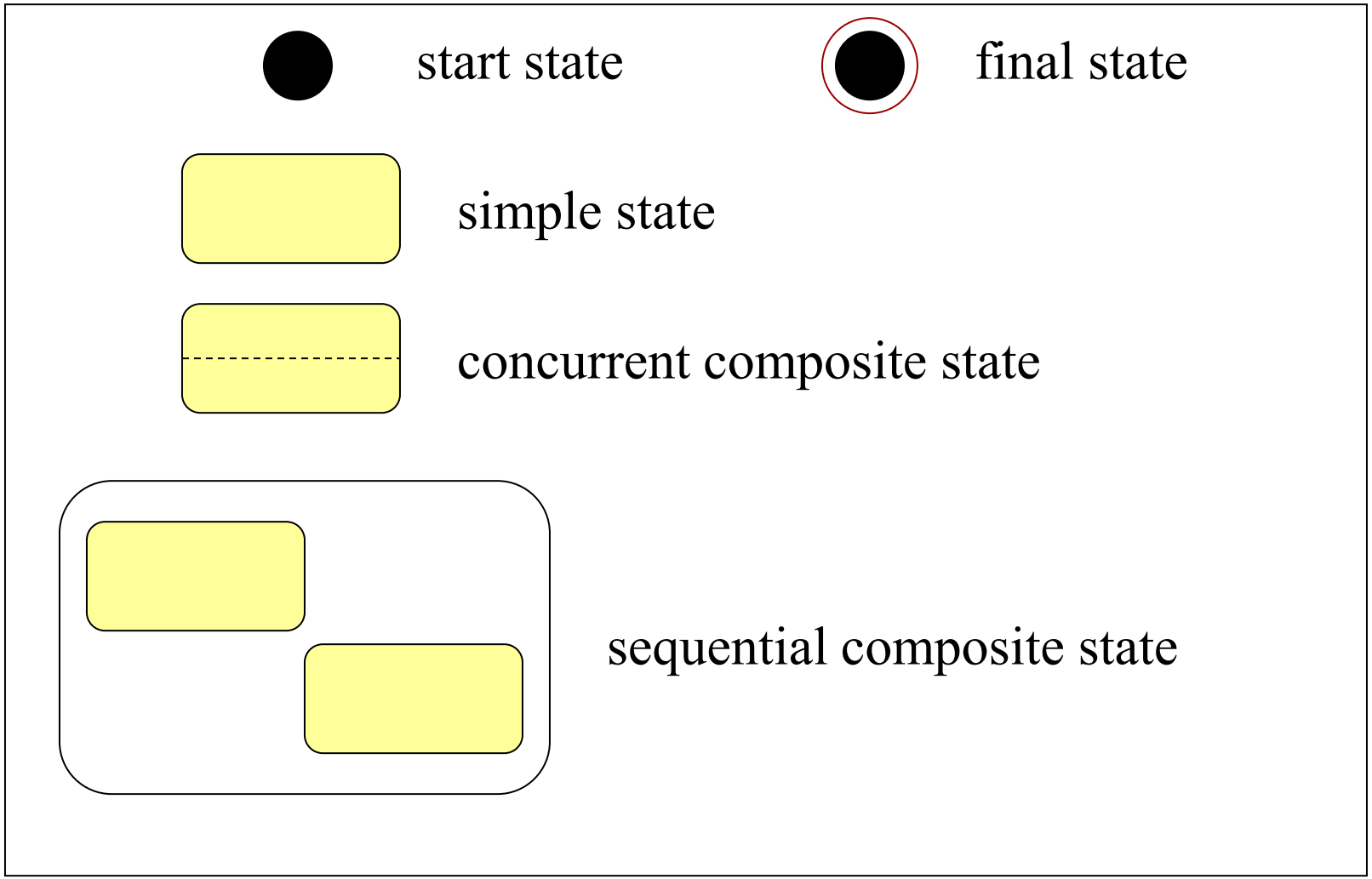
- *The UML Reference Manual, [Rumbaugh,99]*

State Machine

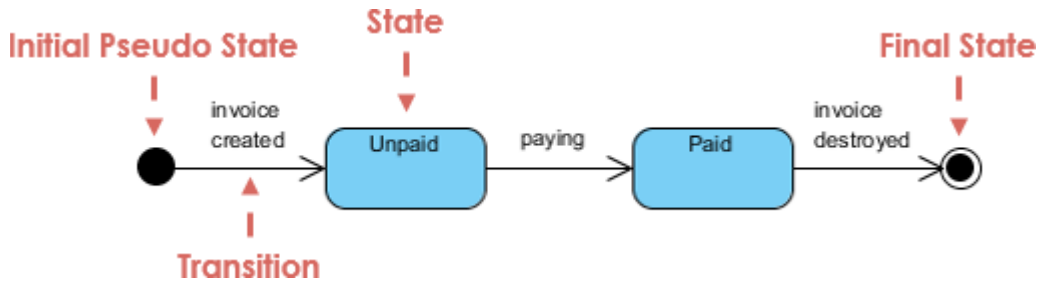
An object must be in some specific state at any given time during its lifecycle. An object transitions from one state to another as the result of some event that affects it. You may create a state diagram for any class, collaboration, operation, or use case in a UML model .

There can be only one start state in a state diagram, but there may be many intermediate and final states.

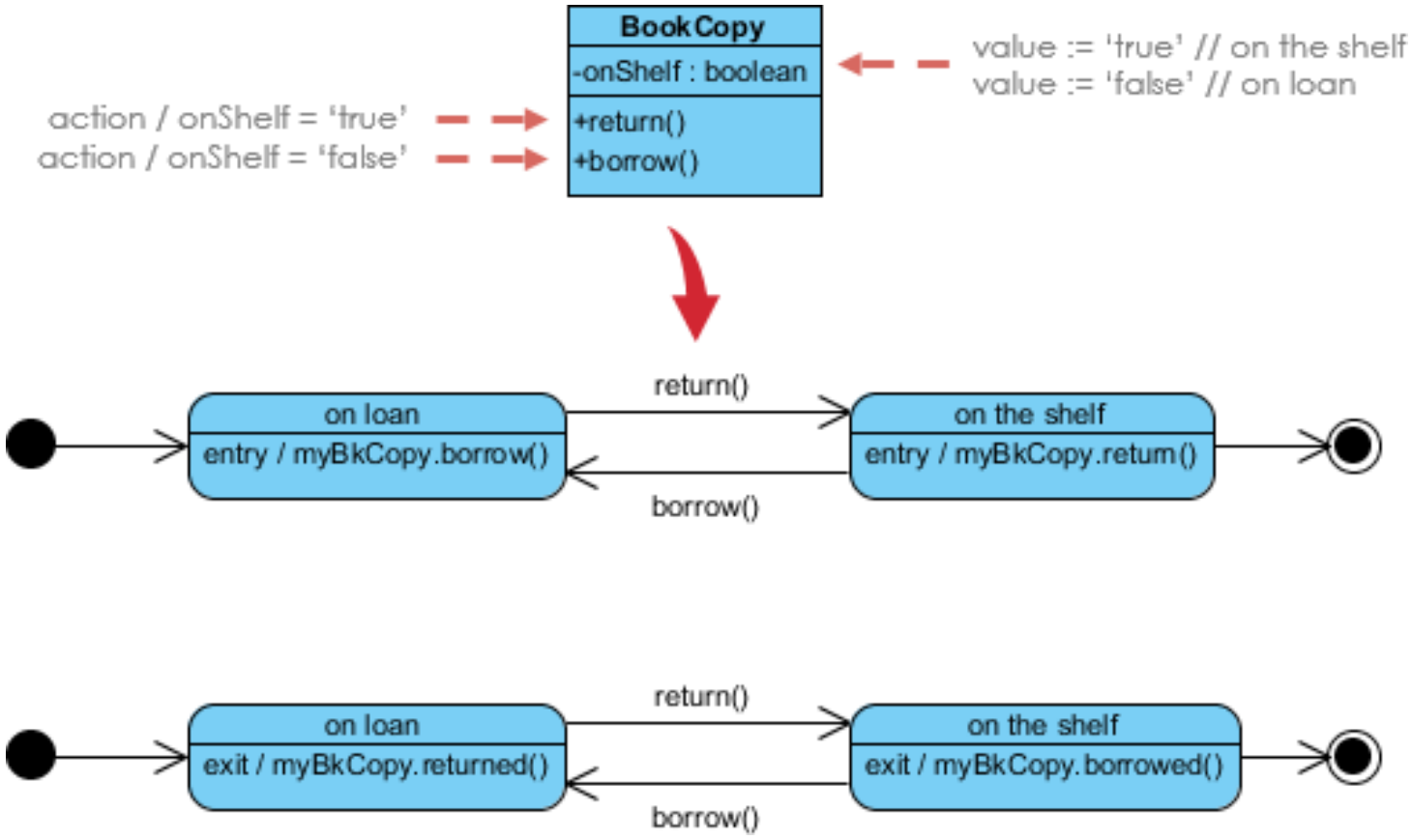
State Machine



State Machine (Contd.)



State Machine (Contd.)



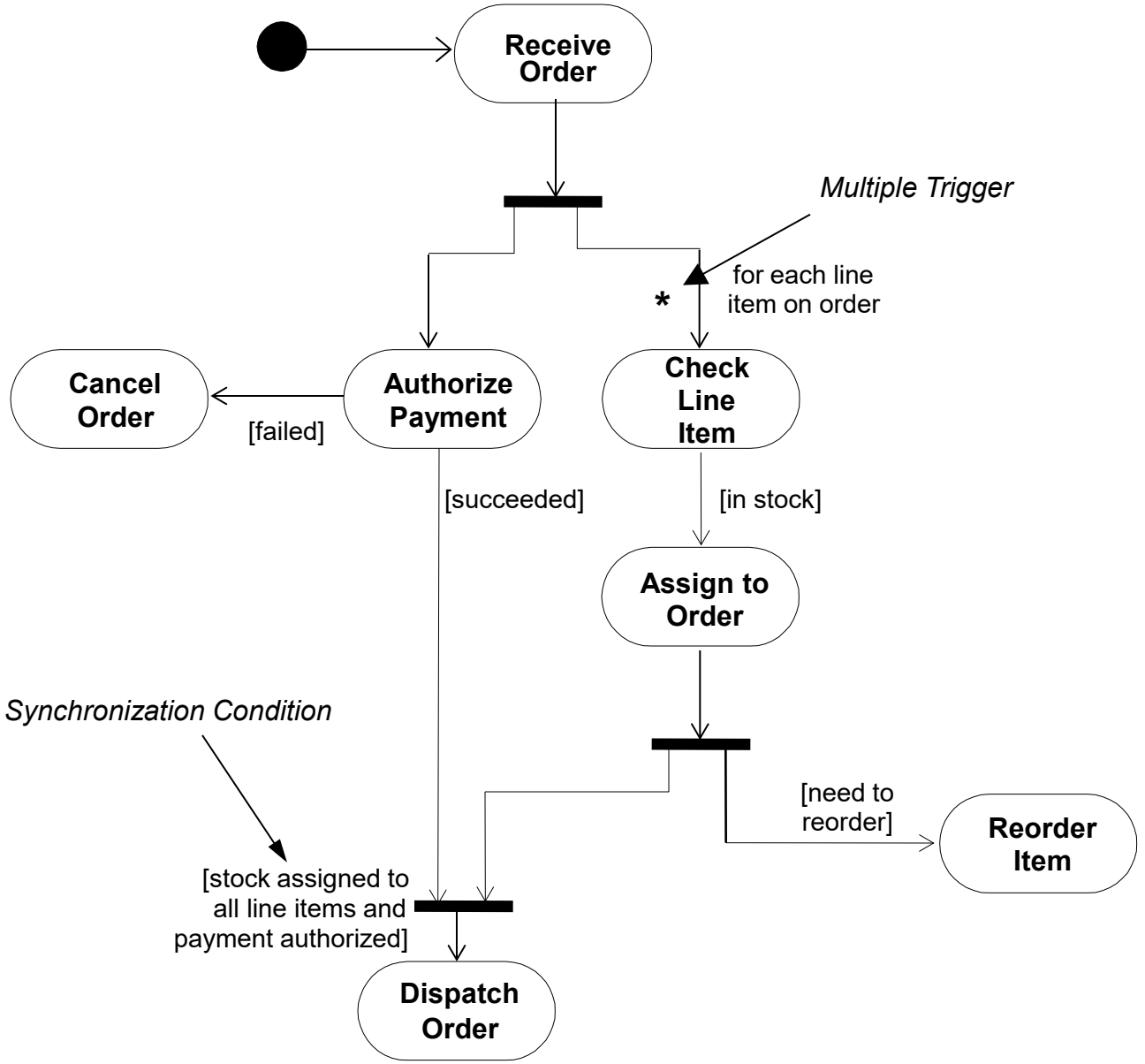
Activity Diagram

An activity diagram is essentially a flowchart, showing the flow of control from activity to activity.

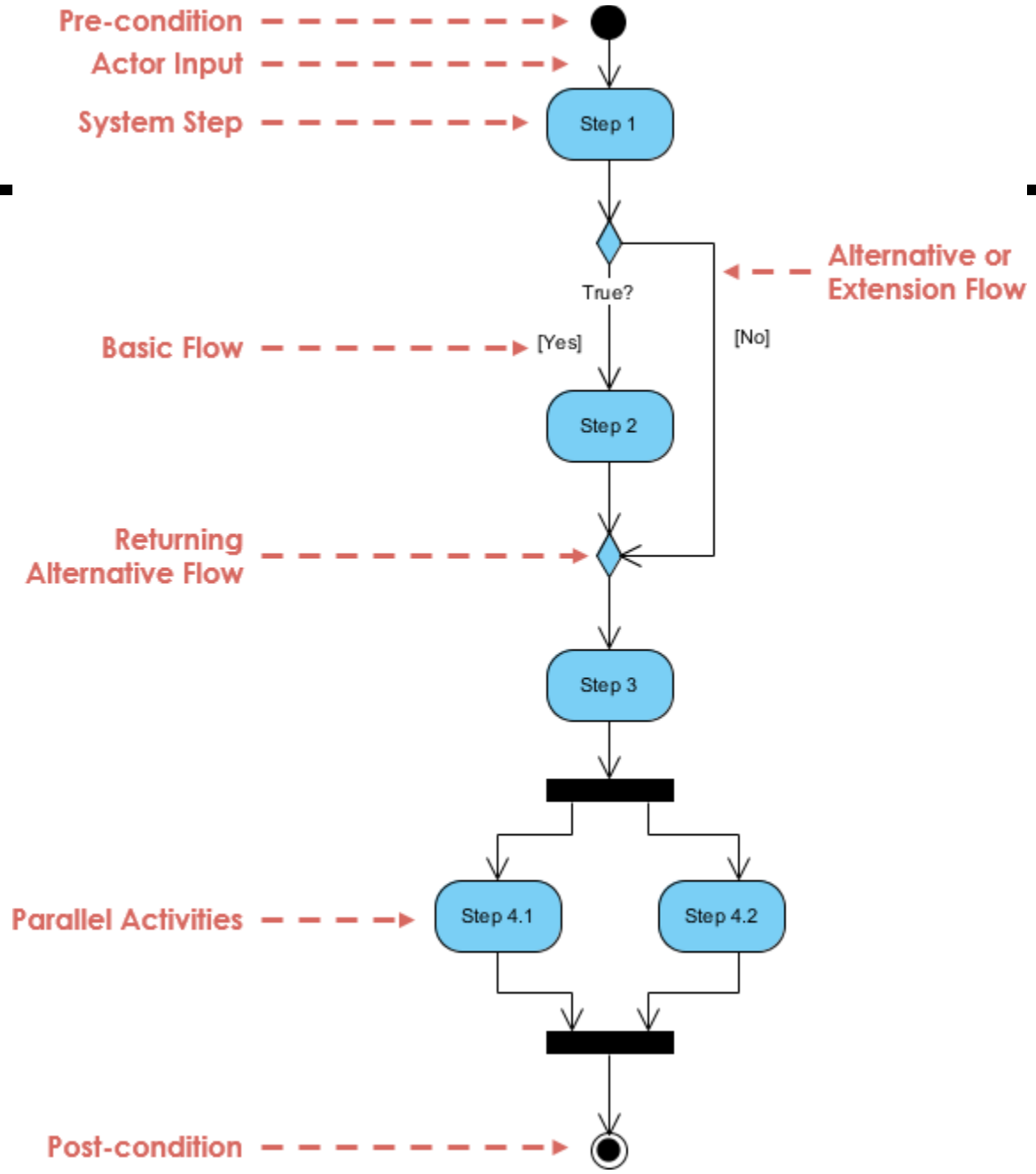
Use activity diagrams to specify, construct, and document the dynamics of a society of objects, or to model the flow of control of an operation. Whereas interaction diagrams emphasize the flow of control from object to object, activity diagrams emphasize the flow of control from activity to activity. ***An activity is an ongoing non-atomic execution within a state machine.***

- *The UML User Guide, [Booch,99]*

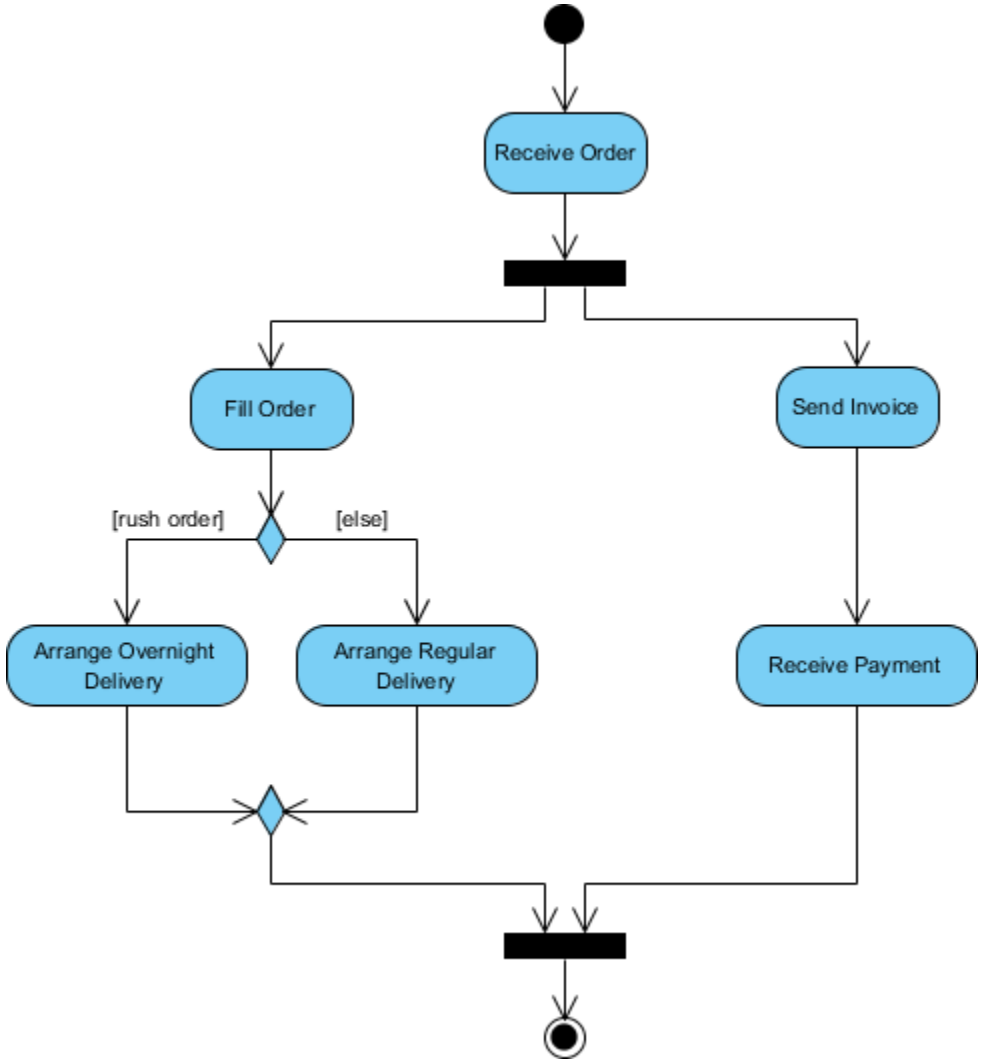
[Fowler,97]


Receive Order
Multiple Trigger
for each line item on order
*
Cancel Order
Authorize Payment
Check Line Item
[failed]
[succeeded]
[in stock]
Assign to Order
Synchronization Condition
[need to reorder]
Reorder Item
[stock assigned to all line items and payment authorized]
Dispatch Order

Example


Pre-condition
Actor Input
System Step
Step 1
True?
Alternative or
Extension Flow
Basic Flow
[Yes]
[No]
Step 2
Returning
Alternative Flow
Step 3
Parallel Activities
Step 4.1
Step 4.2
Post-condition

Example


Receive Order
Fill Order
Send Invoice
[rush order]
[else]
Arrange Overnight Delivery
Arrange Regular Delivery
Receive Payment